



**Politécnico
de Viseu**

Escola Superior
de Tecnologia
e Gestão de Viseu

Intervenção Social Comunitária

Rodrigo Ribeiro Campos Santiago

Trabalho de Projeto

Licenciatura em

Engenharia Informática

Abril de 2026



**Politécnico
de Viseu**

Escola Superior
de Tecnologia
e Gestão de Viseu



departamento de
informática

Relatório do Website Intervenção Social Comunitária

Rodrigo Ribeiro Campos Santiago

Realizado no Solar dos Peixotos (Viseu)

Relatório de Projeto

Licenciatura em

Engenharia Informática

Trabalho efetuado sob a orientação de
José Francisco Monteiro Morgado (ESTGV)
Maria Fernanda Rodrigues de Almeida (Solar dos
Peixotos)

Abril de 2026

Dedicatória

Dedico este esforço e trabalho, a alguém muito especial da minha família, que sempre acreditaram em mim, deram-me apoio e fizeram sacrifícios pela minha saúde. E espero ter-lhes retribuído sempre da melhor maneira possível.

Aos meus pais.

Agradecimentos

Quero agradecer a todos os que me acompanharam neste trajeto, que sofreram, que me deram alento e ânimo para percorrer a encosta que leva ao cimo da montanha.

Ao Professor José Francisco Monteiro Morgado, pelo apoio e coordenação no desenvolvimento deste Projeto Curricular.

À orientadora Maria Fernanda Rodrigues de Almeida pela ajuda e acompanhamento ou esclarecimento de dúvidas durante a execução do Projeto em questão.

Ao Solar dos Peixotos, parte da Administração, pelo ambiente confortável de trabalho e pelas pessoas simpáticas e prestativas para com o Estagiário.

Resumo

O relatório relata o desenvolvimento do projeto ou trabalho da cadeira de estágio curricular realizada na Licenciatura de Engenharia Informática, e na Escola Superior de Tecnologia e Gestão de Viseu do Instituto Politécnico de Viseu. Durante o segundo semestre do terceiro ano, o estágio consistiu na criação de uma plataforma web com o objetivo de gestão de intervenção social comunitária, ou seja, ao atendimento de utentes com dificuldades socioeconómicas e utentes idosos.

O trabalho realizado incluiu, primeiramente, a análise de requisitos necessários para a plataforma, a implementação da base de dados SQL, desenvolvimento do backend em Python, Django e PostgreSQL, e desenvolvimento do frontend com recurso a ferramentas como Bootstrap e JavaScript. Posteriormente, procedeu-se ao desenvolvimento de módulos de criação, listagem, consulta, edição e eliminação de utentes, intervenções, encaminhamentos e contactos, tal como as funcionalidades de geração de relatórios de dados importantes e de dashboard interativo.

A estrutura do relatório consiste em uma introdução a este projeto e relatório desenvolvidos, descrição detalhada sobre a implementação da plataforma web, as principais funcionalidades implementadas detalhadamente, os testes realizados à plataforma. E por fim, as conclusões deste relatório.

Concluiu-se que o estágio cumpriu o objetivo de informatizar o trabalho da intervenção social comunitária do Solar dos Peixotos, proporcionando uma gestão mais eficiente e informativa dos dados administrativos e apoiando a decisão de práticas estratégicas.

PALAVRAS-CHAVE: Desenvolvimento Web, Análise de Dados SQL, Apoio Social

Índice

ÍNDICE DE FIGURAS.....	VIII
ÍNDICE DE TABELAS.....	X
SIGLAS.....	XI
1 INTRODUÇÃO	1
1.1 ENTIDADE DE ACOLHIMENTO.....	1
1.2 OBJETIVOS E RESULTADOS ESPERADOS.....	3
1.3 PLANO DE TRABALHO	4
1.4 ESTRUTURA DO RELATÓRIO	5
2 ESTADO DA ARTE	7
3 METODOLOGIAS, TECNOLOGIAS E FERRAMENTAS UTILIZADAS	9
3.1 METODOLOGIA DE DESENVOLVIMENTO	11
3.2 TECNOLOGIAS DE DESENVOLVIMENTO	13
3.2.1 <i>Backend Web</i>	13
3.2.2 <i>Base de Dados</i>	15
3.2.3 <i>Frontend</i>	15
3.2.4 <i>Geração de Relatórios</i>	16
3.2.5 <i>Ferramentas Utilizadas</i>	17
4 ATIVIDADES DESENVOLVIDAS.....	18
4.1 ANÁLISE DE REQUISITOS.....	19
4.2 MODELAÇÃO DA BASE DE DADOS.....	19
4.3 IMPLEMENTAÇÃO DO BACKEND	29
4.3.1 <i>Arquitetura Geral</i>	30
4.3.2 <i>Modelação da Base de Dados</i>	31
4.3.3 <i>Implementação dos modelos Django</i>	33
4.3.4 <i>Views (Lógica de negócio)</i>	33
4.3.5 <i>Integração com o Frontend</i>	57
5 RESULTADOS OBTIDOS.....	59
5.1 SÍNTESE DOS RESULTADOS ALCANÇADOS.....	59
5.2 PROPOSTAS DE MELHORIA PARA A ORGANIZAÇÃO	59
6 CONCLUSÃO.....	61

6.1	PRINCIPAIS RESULTADOS E CONTRIBUIÇÕES	61
6.2	TRABALHO FUTURO	61
6.3	APRENDIZAGEM E REFLEXÃO PESSOAL	62
REFERÊNCIAS BIBLIOGRÁFICAS		63

Índice de Figuras

FIGURA 1 - PRIMEIRA PARTE DO MODELO CONCEPTUAL DE DADOS (POWERDESIGNER)	20
FIGURA 2 - SEGUNDA PARTE DO MODELO CONCEPTUAL DE DADOS (POWERDESIGNER)	21
FIGURA 3 - ÚLTIMA PARTE DO MODELO CONCEPTUAL DE DADOS (POWERDESIGNER)	21
FIGURA 4 - PRIMEIRA PARTE DO MODELO LÓGICO DE DADOS (POWERDESIGNER)	22
FIGURA 5 - SEGUNDA PARTE DO MODELO LÓGICO DE DADOS (POWERDESIGNER)	22
FIGURA 6 - ÚLTIMA PARTE DO MODELO LÓGICO DE DADOS (POWERDESIGNER)	23
FIGURA 7 - PRIMEIRA PARTE DO MODELO FÍSICO DE DADOS (POWERDESIGNER)	23
FIGURA 8 - SEGUNDA PARTE DO MODELO FÍSICO DE DADOS (POWERDESIGNER)	24
FIGURA 9 - ÚLTIMA PARTE DO MODELO FÍSICO DE DADOS (POWERDESIGNER)	24
FIGURA 10 - SECÇÃO DOS DROP TABLES DO SCRIPT DE CRIAÇÃO DE TABELAS (DDL)	25
FIGURA 11 - CRIAÇÃO DE TABELAS DA BASE DE DADOS	25
FIGURA 12 - CRIAÇÃO DE TABELAS AUXILIARES	26
FIGURA 13 - CRIAÇÃO DAS TABELAS UTENTES E UTENTES IDOSOS	26
FIGURA 14 - CRIAÇÃO DA TABELA CONTACTOS DE IDOSOS	27
FIGURA 15 - CRIAÇÃO DA TABELA UTENTES SOCIOECONÓMICOS	27
FIGURA 16 - CRIAÇÃO DA TABELA INTERVENÇÃO	28
FIGURA 17 - CRIAÇÃO DA TABELA ENCAMINHAMENTOS	28
FIGURA 18 - CRIAÇÃO DE OUTRAS TABELAS	29
FIGURA 19 - VISTA EM SQL DOS NÚMEROS TOTAIS DOS UTENTES, INTERVENÇÕES E CONTACTOS REGISTADOS	32
FIGURA 20 - VISTA DO TOTAL DE INTERVENÇÕES POR MÊS E ANO	32
FIGURA 21 - VISTA DOS IDOSOS COM NÍVEL DE ISOLAMENTO ELEVADO	33
FIGURA 22 - MODELO UTENTES EM PYTHON (MODELS.PY)	33
FIGURA 23 - VIEWS DE LOGIN E LOGOUT	34
FIGURA 24 - VISTA DE LISTAGEM DE UTENTES SOCIOECONÓMICOS	35
FIGURA 25 - VISTA DE DETALHAR UM UTENTE (DADOS PRINCIPAIS)	36
FIGURA 26 - VISTA DE DETALHAR UM UTENTE (INTERVENÇÕES E PROBLEMÁTICAS)	36
FIGURA 27 - VISTA DE DETALHAR UM UTENTE (PARTE FINAL)	37
FIGURA 28 - PRIMEIRA PARTE DA VIEW DE CRIAÇÃO	38
FIGURA 29 - SEGUNDA PARTE DA VIEW DE CRIAÇÃO	38
FIGURA 30 - TERCEIRA PARTE DA VIEW DE CRIAÇÃO	39
FIGURA 31 - QUARTA PARTE DA VIEW DE CRIAÇÃO	39

FIGURA 32 - QUINTA PARTE DA VIEW DE CRIAÇÃO.....	40
FIGURA 33 - PARTE FINAL DA VIEW DE CRIAÇÃO.....	40
FIGURA 34 - PRIMEIRA PARTE DA VIEW DE EDIÇÃO	41
FIGURA 35 - SEGUNDA PARTE DA VIEW DE EDIÇÃO	41
FIGURA 36 - TERCEIRA PARTE DA VIEW DE EDIÇÃO	42
FIGURA 37 - QUARTA PARTE DA VIEW DE EDIÇÃO.....	42
FIGURA 38 - QUINTA PARTE DA VIEW DE EDIÇÃO.....	43
FIGURA 39 - SEXTA PARTE DA VIEW DE EDIÇÃO	43
FIGURA 40 - PARTE FINAL DA VIEW DE EDIÇÃO	44
FIGURA 41 - VIEW DE ELIMINAR	44

Índice de Tabelas

TABELA 1 - ARQUITETURA MTV DO DJANGO	31
TABELA 2 - TABELAS DA BASE DE DADOS SQL	32

Siglas

AJAX – Asynchronous JavaScript and XML

AVD – Atividades da Vida Diária

CRUD – Create, Read, Update, Delete

CSS – Cascading Style Sheets

DDL – Data Definition Language

DOM – Document Object Model

ESTGV – Escola Superior de Tecnologia e Gestão de Viseu

HTML – HyperText Markup Language

HTTP – HyperText Transfer Protocol

IPV – Instituto Politécnico de Viseu

JSON – JavaScript Object Notation

JSONB – JavaScript Object Notation Binary

MTV – Model-Template-View

ORM – Object-Relational Mapping

PDF – Portable Document Format

RGPD – Regulamento Geral sobre a Proteção de Dados

SQL – Structured Query Language

URL – Uniform Resource Locator

1 Introdução

O presente relatório descreve o trabalho desenvolvido no âmbito do estágio curricular da Licenciatura em Engenharia Informática, realizado na Escola Superior de Tecnologia e Gestão de Viseu do Instituto Politécnico de Viseu. O estágio decorreu no segundo semestre do terceiro ano letivo 2025/2026, entre Fevereiro e Junho de 2026, tendo como entidade de acolhimento o Solar dos Peixotos, em Viseu. Esta instituição, instalada num edifício de reconhecido valor histórico e arquitetónico, presta apoio a utentes com dificuldades socioeconómicas e a utentes idosos.

O projeto consistiu no desenvolvimento de uma plataforma web para a gestão da intervenção social comunitária, evidenciando a importância da Engenharia Informática na digitalização e otimização dos serviços de apoio social. A plataforma visa substituir os métodos tradicionais baseados em papel e folhas de cálculo, oferecendo uma solução integrada que permite o registo, acompanhamento e análise de utentes, intervenções, encaminhamentos e contactos. Através da centralização da informação, da automatização de processos e da geração de indicadores de gestão, a plataforma contribui para uma resposta mais ágil, eficaz e fundamentada da equipa de intervenção social.

A unidade curricular de estágio tem como propósito proporcionar aos estudantes a oportunidade de integrar um ambiente profissional real, aplicando os conhecimentos adquiridos ao longo do curso e adquirindo experiência prática relevante para o mercado de trabalho. Durante este período, foram desenvolvidas competências técnicas e transversais, nomeadamente ao nível da análise de requisitos, conceção de bases de dados, desenvolvimento backend e frontend, controlo de versões, trabalho com tecnologias web e comunicação com os stakeholders. O estágio constituiu uma experiência formativa de grande valor, consolidando os conceitos teóricos aprendidos na licenciatura e expondo o estagiário aos desafios reais da engenharia de software em contexto organizacional.

1.1 Entidade de Acolhimento

O estágio foi realizado na Junta de Freguesia de Viseu, com sede no Solar dos Peixotos, em Viseu. Este edifício, um notável exemplar da arquitetura senhorial barroca do século XVIII, foi mandado construir em 1729 pelo cônego da Sé de Viseu, Nicolau de Almeida Castelo-Branco. Posteriormente, foi adquirido pelo seu primo, António de Loureiro Castelo-Branco de Vasconcelos, permanecendo na posse da família até 1962. Após uma breve transição de propriedade, o imóvel foi adquirido pelo Município de Viseu durante o século XX. Entre 2018 e 2020, foi alvo de um extenso projeto de requalificação que modernizou as infraestruturas elétricas, de telecomunicações e de aquecimento, tendo também sido instalado um elevador. Atualmente, o Solar dos Peixotos alberga a nova sede da Junta de Freguesia e da Assembleia Municipal de Viseu, constituindo um espaço de referência para a administração local e para o atendimento aos cidadãos.

A Junta de Freguesia de Viseu tem como missão assegurar a execução de políticas públicas de proximidade, promovendo o bem-estar da população e a coesão social. As suas atividades principais inserem-se nas áreas da Administração Pública, Defesa e Segurança Social obrigatória, abrangendo serviços como o registo civil, o licenciamento, a gestão de espaços públicos e a emissão de documentos administrativos. Para além destas competências formais, a entidade desenvolve ainda atividades secundárias de grande relevância social, com destaque para um intenso trabalho de apoio social à comunidade. Este apoio traduz-se no acompanhamento de famílias em situação de vulnerabilidade, na atribuição de apoios alimentares e medicamentosos, no encaminhamento para serviços especializados e na promoção de iniciativas de inclusão e solidariedade.

A crescente procura por estes serviços, aliada à complexidade dos problemas socioeconómicos e ao envelhecimento populacional, evidenciou a necessidade de ferramentas informáticas que permitissem uma gestão mais eficiente e transparente da intervenção social. Foi neste contexto que surgiu a oportunidade de desenvolver a plataforma web descrita neste relatório, com o propósito de digitalizar processos, centralizar a informação e melhorar a resposta da Junta de Freguesia às necessidades da sua população.

Para cumprir os seus objetivos, a entidade dispõe de uma equipa de Recursos Humanos dedicada às funções administrativas e de gestão, bem como de técnicos de intervenção social que acompanham diariamente dezenas de utentes. A colaboração entre o estagiário e estes profissionais foi fundamental para o levantamento de requisitos, a validação das funcionalidades implementadas e a garantia de que a plataforma respondesse efetivamente às necessidades do terreno. O envolvimento direto da equipa permitiu ainda identificar indicadores de desempenho e, também, uma gestão proativa dos casos mais críticos, como

situações de isolamento elevado, quedas repetidas ou falta de contacto com idosos. Assim, a entidade de acolhimento proporcionou um ambiente rico em aprendizagem e um alinhamento próximo entre a solução tecnológica e a realidade da intervenção social comunitária.

1.2 Objetivos e Resultados Esperados

O estágio realizado no Solar dos Peixotos teve como propósito central modernizar o atendimento e fortalecer a resposta social da Junta de Freguesia de Viseu aos seus cidadãos, especialmente àqueles em situação de vulnerabilidade socioeconómica e aos utentes idosos. Para tal, foram definidos objetivos específicos que orientaram todo o trabalho desenvolvido, bem como um conjunto de resultados esperados que permitem avaliar o impacto da plataforma na atividade da instituição. Um dos principais objetivos consistiu em informatizar o apoio prestado à população, substituindo os processos manuais (baseados em papel, folhas de cálculo e registos dispersos) por uma solução digital integrada, capaz de agilizar o registo, a consulta e a atualização de informação sobre utentes, intervenções, encaminhamentos e contactos. Esta transformação digital visava reduzir a redundância administrativa, minimizar erros de registo e libertar os técnicos para atividades de maior valor acrescentado.

Paralelamente, pretendeu-se reforçar a coesão social através da implementação de programas de apoio estruturados, permitindo identificar e intervir junto das situações de maior vulnerabilidade de forma mais eficaz. A plataforma foi concebida para apoiar a gestão de problemáticas (como isolamento social, carência alimentar, endividamento, violência doméstica, saúde mental, entre outras) e para monitorizar a evolução de cada utente ao longo do tempo, incluindo a associação de intervenções à resolução de problemáticas. Como consequência direta destas linhas de ação, ambicionou-se melhorar a eficiência e a qualidade da gestão da intervenção social comunitária em Viseu, proporcionando às equipas técnicas ferramentas que facilitassem o planeamento, o acompanhamento e a tomada de decisão fundamentada. Para materializar estes objetivos, foi desenvolvida uma plataforma web dedicada ao registo, monitorização e análise de dados de utentes em situação de vulnerabilidade socioeconómica, bem como de utentes idosos, assegurando uma visão integrada e contínua do seu percurso de apoio social. A plataforma inclui ainda funcionalidades de geração de relatórios automáticos (mensais e anuais) e um dashboard com indicadores de gestão (evolução de problemáticas, taxa de reincidência, idosos isolados, quedas repetidas, visitas prioritárias, etc.), permitindo uma avaliação sistemática da eficácia das intervenções.

Com a implementação da plataforma, esperava-se obter vários resultados concretos. Em primeiro lugar, a centralização e fiabilidade da informação, com todos os registos relativos a utentes, intervenções, encaminhamentos e contactos disponíveis numa única base de dados, acessível remotamente e com controlo de acessos, reduzindo a perda de informação e a duplicação de esforços. Em segundo lugar, a redução do tempo de resposta, uma vez que a pesquisa por nome, a paginação e os filtros nas listagens permitem localizar um utente ou uma intervenção em poucos segundos, ao contrário dos métodos manuais anteriores, prevendo-se uma redução significativa do tempo despendido em tarefas administrativas. Em terceiro lugar, a melhoria da capacidade de monitorização através do dashboard e das estatísticas automáticas (ex: idosos sem contacto há mais de 4 semanas, quedas repetidas, isolamento elevado, agravamento da dependência), permitindo à equipa identificar rapidamente situações críticas e priorizar visitas ou medidas de prevenção. A plataforma visa ainda apoiar a decisão estratégica, fornecendo relatórios PDF (mensais e anuais) com dados agregados sobre perfis de utentes, problemáticas mais frequentes, tipos de intervenção, programas utilizados, respostas de encaminhamentos e taxas de reincidência, fundamentais para candidaturas a financiamento, justificação de recursos humanos e elaboração de relatórios para entidades tutelares. Por fim, espera-se que a plataforma contribua para a rastreabilidade e conformidade, através do sistema de histórico de alterações (snapshots JSON) que garante a rastreabilidade de todas as modificações efetuadas nos registos de utentes, permitindo auditorias e a recuperação de informação em caso de erro, em conformidade com as boas práticas de proteção de dados. No final do estágio, os resultados esperados foram, na sua maioria, alcançados, tendo a plataforma sido validada pela entidade de acolhimento e considerada apta para utilização em ambiente real. Algumas funcionalidades avançadas (como integração com sistemas externos ou aplicação móvel) ficaram como propostas de trabalho futuro, mas a base está sólida e funcional.

1.3 Plano de Trabalho

Durante o período de estágio, o plano de trabalho foi organizado em quatro fases sequenciais, de modo a garantir uma evolução coerente e consistente do projeto.

Na primeira fase, procedeu-se à análise detalhada dos requisitos específicos para a plataforma. Esta etapa envolveu reuniões com os técnicos de intervenção social, a observação dos processos manuais em uso e a identificação das principais necessidades funcionais, tais como o registo de utentes, o acompanhamento de intervenções e a produção

de indicadores de gestão. O resultado desta fase foi um documento de requisitos que serviu de base para todo o desenvolvimento.

Na segunda fase, dedicou-se à criação e desenvolvimento da base de dados relacional do projeto. Foram identificadas todas as entidades relevantes (utentes socioeconómicos, idosos, intervenções, encaminhamentos, contactos, problemáticas, etc.) e estabelecidas as relações entre elas, garantindo a normalização e a integridade referencial. Paralelamente, definiu-se a arquitetura tecnológica, optando-se pelo framework Django (Python) para o backend e pelo PostgreSQL como sistema de gestão de base de dados, assegurando robustez, segurança e escalabilidade.

A terceira fase consistiu no desenvolvimento da plataforma web propriamente dita. Foram implementados os módulos de gestão de utentes (socioeconómicos e idosos), intervenções, encaminhamentos, contactos, bem como um dashboard com indicadores analíticos, um sistema de histórico de alterações (com recurso a snapshots JSON) e a geração de relatórios em formato PDF, mensais e anuais, com filtragem temporal.

Por fim, na quarta fase, realizaram-se testes de validação e correção de erros, com base em dados reais fornecidos pela entidade de acolhimento. Foram também elaborados os manuais de utilizador e de administração, e redigido o relatório final de estágio, que descreve de forma exaustiva todo o trabalho realizado, as dificuldades encontradas e as soluções adotadas.

1.4 Estrutura do Relatório

A estrutura do relatório encontra-se organizada em seis capítulos principais, os quais refletem a sequência lógica do trabalho desenvolvido.

O Capítulo 1 apresenta uma caracterização fundamentada da entidade de acolhimento – a Junta de Freguesia de Viseu, sediada no Solar dos Peixotos – e descreve os objetivos gerais da plataforma desenvolvida no âmbito do estágio.

O Capítulo 2 apresenta o enquadramento teórico do projeto, expondo as principais fontes académicas e documentais que fundamentaram o desenvolvimento, as disciplinas do curso que contribuíram para o planeamento das atividades e as tecnologias estudadas de forma autónoma ao longo do estágio.

O Capítulo 3 expõe as opções de arquitetura e as tecnologias adotadas, justificando a escolha do Django como framework backend, do PostgreSQL como sistema de gestão de base de

dados, do Bootstrap para a interface gráfica e das restantes bibliotecas e componentes utilizados.

O Capítulo 4 constitui o núcleo do relatório, detalhando o processo de implementação de cada módulo funcional: gestão de utentes socioeconómicos, gestão de utentes idosos, intervenções, encaminhamentos, contactos, desenvolvimento do dashboard analítico, sistema de histórico de alterações e geração de relatórios PDF.

O Capítulo 5 apresenta os resultados obtidos, sintetizando os principais contributos da plataforma para a entidade de acolhimento e as propostas de melhoria identificadas ao longo do estágio.

Por fim, o Capítulo 6 expõe as conclusões do trabalho, destacando os principais resultados e contribuições, as aprendizagens adquiridas ao longo do estágio e as perspetivas de desenvolvimento futuro da plataforma.

2 Estado da Arte

A base teórica para o desenvolvimento do projeto estruturou-se, em grande medida, nas competências adquiridas ao longo da Licenciatura em Engenharia Informática, com particular ênfase nas unidades curriculares de Bases de Dados e de Aplicações para Internet. A primeira proporcionou os fundamentos da modelação conceptual e relacional, a normalização, a integridade referencial e a otimização de consultas SQL, conceitos que foram essenciais para a conceção do esquema da base de dados subjacente à plataforma, garantindo a consistência e a eficiência no armazenamento e recuperação da informação. A disciplina de Aplicações para Internet, por sua vez, dotou o estagiário dos princípios de desenvolvimento de aplicações web, nomeadamente a arquitetura cliente-servidor, o protocolo HTTP, o tratamento de formulários, a gestão de sessões e a construção de interfaces dinâmicas, conhecimentos que foram diretamente aplicados na implementação da plataforma.

Para além dos conteúdos lecionados, foi realizada uma extensa pesquisa autónoma que permitiu aprofundar as tecnologias específicas utilizadas no projeto. A documentação oficial do Django foi uma fonte essencial para compreender o seu sistema de modelos (ORM), a gestão de autenticação, as vistas baseadas em classes e a integração com o PostgreSQL, contribuindo para a construção de um backend robusto e seguro. Quanto ao sistema de gestão de base de dados, recorreu-se à documentação do PostgreSQL para explorar funcionalidades como a utilização de índices, a otimização de consultas com “EXPLAIN”, o tipo de dados JSONB (utilizado para armazenar snapshots de histórico) e a criação de vistas materializadas para simplificar consultas analíticas complexas. A biblioteca ReportLab foi estudada para permitir a geração dinâmica de documentos PDF em ambiente Django, tendo-se aprendido a construir layouts com tabelas, parágrafos estilizados, quebras de página e formatação de datas, o que foi fundamental para a criação de relatórios mensais e anuais completamente parametrizáveis. A biblioteca Chart.js, por seu turno, foi utilizada para a visualização de dados no dashboard, tendo sido necessário compreender a passagem de dados do backend para o frontend em formato JSON, bem como a configuração de gráficos interativos de linhas, barras e pizza.

Adicionalmente, recorreu-se a ferramentas de Inteligência Artificial, nomeadamente o ChatGPT e o DeepSeek, para complementar a aprendizagem teórica e auxiliar na resolução de obstáculos pontuais durante o desenvolvimento. Estas ferramentas foram utilizadas para gerar explicações alternativas sobre conceitos complexos, sugerir correções para erros de sintaxe ou lógica em SQL e Python, e propor abordagens para a construção de consultas analíticas (ex: cálculo de evolução de problemáticas e taxas de reincidência). Sempre que se revelaram úteis, as soluções propostas foram analisadas, adaptadas e integradas criticamente no projeto, após validação e compreensão do seu funcionamento. Desta forma, a base teórica e o suporte de fontes externas contribuíram decisivamente para o planeamento, a execução e a resolução de problemas ao longo de todo o estágio.

3 Metodologias, Tecnologias e Ferramentas Utilizadas

No presente capítulo, descrevem-se as metodologias, tecnologias e ferramentas utilizadas para o desenvolvimento da plataforma web de intervenção social comunitária. A seleção destas opções resultou de uma análise detalhada dos requisitos funcionais e não funcionais da plataforma, bem como da necessidade de garantir uma solução robusta, escalável e de fácil manutenção, capaz de responder às necessidades atuais e futuras da entidade de acolhimento. Foram considerados critérios como o desempenho, a segurança, a facilidade de integração com outros sistemas, a curva de aprendizagem e o suporte da comunidade. Assim, optou-se por uma arquitetura baseada no padrão MTV (Model-Template-View) do Django, complementada por componentes de frontend que asseguram uma experiência de utilizador responsiva e intuitiva. As escolhas tecnológicas e metodológicas apresentadas nos subcapítulos seguintes refletem um equilíbrio entre boas práticas de engenharia de software, agilidade de desenvolvimento e adaptação ao contexto específico da intervenção social.

A decisão de adotar o Django como framework principal deveu-se a vários fatores determinantes. Em primeiro lugar, a sua integração nativa com o PostgreSQL, através do ORM (Object-Relational Mapper), facilita a gestão da base de dados e a execução de consultas complexas, que são frequentes no contexto da plataforma (especialmente para o dashboard e relatórios). Em segundo lugar, o sistema de autenticação e autorização do Django, robusto e extensível, permite implementar facilmente o controlo de acessos necessário para garantir a confidencialidade dos dados, um requisito crítico no âmbito da intervenção social e da proteção de dados. Em terceiro lugar, a vasta documentação e a comunidade ativa em torno do Django proporcionam um suporte valioso, acelerando o ciclo de desenvolvimento e a resolução de problemas.

A escolha do PostgreSQL como sistema de gestão de base de dados, por sua vez, fundamenta-se na sua capacidade de lidar com consultas analíticas complexas (agregações, funções de janela, subconsultas) e no suporte nativo ao tipo de dados JSONB, utilizado para

armazenar os snapshots de histórico dos utentes. Esta capacidade revelou-se essencial para a implementação do sistema de histórico de alterações e para a geração de relatórios com precisão histórica, uma vez que permite guardar e consultar o estado completo de um utente em qualquer momento passado. O PostgreSQL foi ainda escolhido pela sua conformidade com os padrões SQL, pela fiabilidade e pela experiência prévia do estagiário com este sistema, fatores que contribuíram para um desenvolvimento mais ágil e seguro.

No frontend, a utilização do Bootstrap 5 permitiu construir uma interface responsiva e acessível, garantindo que a plataforma se adapta automaticamente a diferentes tamanhos de ecrã e dispositivos. A biblioteca de ícones Bootstrap Icons complementa a experiência visual, fornecendo uma comunicação gráfica clara e consistente em toda a aplicação. A integração entre o backend e o frontend é realizada através do motor de templates do Django, que gera dinamicamente as páginas HTML a partir dos dados fornecidos pelas views. Para os gráficos do dashboard, optou-se pela biblioteca Chart.js, que, sendo leve e fácil de integrar, permite visualizar de forma interativa os indicadores de gestão e as evoluções temporais dos dados.

Para os componentes interativos do frontend, como a seleção de utentes nas intervenções ou a escolha de idosos nos contactos, recorreu-se à biblioteca Select2, que transforma os elementos `<select>` em campos de pesquisa com autocomplete, melhorando significativamente a usabilidade quando existem centenas de opções disponíveis. Este cuidado com a experiência do utilizador reflete a preocupação em adequar a plataforma ao ritmo de trabalho e às competências digitais dos técnicos de intervenção social, garantindo que a ferramenta seja um auxílio e não um obstáculo no seu quotidiano.

Do ponto de vista metodológico, adotou-se uma abordagem ágil baseada em sprints semanais, permitindo uma adaptação contínua aos requisitos e um feedback regular da entidade de acolhimento. Cada sprint iniciou-se com o planeamento das tarefas, seguiu-se o desenvolvimento e os testes, e terminou com uma revisão junto do orientador. Esta metodologia revelou-se particularmente adequada a projetos com requisitos em evolução, onde a proximidade com o utilizador final e a iteração rápida são fatores de sucesso. Para a gestão das tarefas e o acompanhamento do progresso, recorreu-se ao Trello, enquanto o controlo de versões foi assegurado pelo sistema Git, com repositório remoto alojado no GitHub Desktop (interface gráfica), que facilitou a gestão de branches, commits e a consistência do código ao longo do projeto.

Para as consultas SQL mais complexas, nomeadamente as que alimentam o dashboard e os relatórios, optou-se por utilizar SQL puro em vez do ORM do Django. Esta decisão, embora

aumente ligeiramente a complexidade do código, proporciona um controlo total sobre a performance e a semântica das consultas, permitindo a utilização de funções de janela, subconsultas e agregações avançadas que seriam difíceis de expressar de forma eficiente com o ORM. As vistas SQL predefinidas (ex: `vw_evolucao_problematicas`, `vw_taxa_reincidencia_por_problematica`, `vw_idosos_isolados`) são utilizadas exclusivamente para leitura e fornecem dados pré-calculados ao dashboard e aos relatórios, reduzindo a carga computacional em cada pedido e garantindo tempos de resposta adequados.

Para a geração de relatórios em PDF, recorreu-se à biblioteca ReportLab, que permite a construção programática de documentos estruturados com tabelas estilizadas, parágrafos e quebras de página. A geração de relatórios mensais e anuais, com filtragem temporal por período, é acionada através de um formulário no dashboard, e o ficheiro resultante é devolvido ao navegador para download com um nome significativo. A flexibilidade do ReportLab, aliada à capacidade de integrar dados dinâmicos provenientes das consultas SQL, permite produzir documentos com a qualidade e a apresentação necessárias para relatórios de gestão e candidaturas a financiamento.

As ferramentas de desenvolvimento utilizadas incluíram o Visual Studio Code como editor de código, com extensões para Python, Django, HTML, CSS e JavaScript, que contribuíram para a produtividade e a qualidade do código. O sistema de controlo de versões Git, com interface gráfica GitHub Desktop, assegurou a rastreabilidade das alterações e a consistência do código. Para a resolução de dúvidas pontuais e a exploração de soluções alternativas, foram utilizadas ferramentas de inteligência artificial como ChatGPT e DeepSeek, que, embora não substituam o conhecimento técnico, serviram como um suporte adicional na depuração de erros e na aprendizagem de novos conceitos.

Em suma, o conjunto de metodologias, tecnologias e ferramentas adotadas revelou-se adequado ao desenvolvimento da plataforma, permitindo cumprir os requisitos funcionais e não funcionais do projeto, respeitar os prazos estabelecidos e garantir a segurança e a usabilidade da aplicação. A combinação de Django, PostgreSQL, Bootstrap e SQL puro, aliada a uma metodologia ágil e a ferramentas de apoio eficientes, proporcionou um equilíbrio entre a produtividade no desenvolvimento e a robustez necessária para uma aplicação de gestão de dados sensíveis em contexto de intervenção social comunitária.

3.1 Metodologia de Desenvolvimento

Adotou-se uma metodologia ágil inspirada no Scrum, estruturada em sprints semanais, o que permitiu uma adaptação contínua aos requisitos, uma resposta rápida a alterações e um feedback regular por parte da entidade de acolhimento. Esta abordagem revelou-se particularmente adequada ao projeto, uma vez que as necessidades da intervenção social comunitária são dinâmicas e frequentemente dependem de contextos emergentes, como a identificação de novas problemáticas ou a necessidade de priorizar determinados grupos de utentes. Cada sprint teve a duração de uma semana e foi dividido em três fases distintas: no início, realizava-se o planeamento das tarefas, com a definição de objetivos e a priorização das funcionalidades a desenvolver; de seguida, procedia-se ao desenvolvimento propriamente dito e à execução dos testes unitários e de integração; por fim, a sprint concluía-se com uma revisão junto do orientador, onde se analisavam os resultados obtidos e se recolhiam sugestões de melhoria para as iterações seguintes.

Esta estrutura cíclica permitiu não só manter o foco nas entregas de valor, mas também incorporar o feedback da equipa de intervenção social de forma sistemática, validando se as funcionalidades implementadas correspondiam efetivamente às necessidades do terreno. Por exemplo, durante as sprints iniciais, a necessidade de um sistema de histórico de alterações e de estatísticas automáticas para situações de risco foi identificada e rapidamente integrada no plano de desenvolvimento, demonstrando a flexibilidade do processo. A abordagem iterativa e incremental revelou-se particularmente adequada ao projeto, uma vez que permitiu ajustar o rumo do desenvolvimento com base na experiência prática e nas necessidades emergentes da equipa de intervenção social, evitando o desperdício de esforço em funcionalidades que, na prática, se revelavam menos prioritárias.

Para a gestão das tarefas e o acompanhamento do progresso, recorreu-se ao Trello, uma ferramenta de gestão de projetos baseada em quadros Kanban, que permitiu visualizar o estado de cada atividade (planeada, em desenvolvimento, em teste, concluída) e priorizar as funcionalidades de acordo com o valor acrescentado para a entidade de acolhimento. A utilização do Trello facilitou ainda a comunicação com o orientador, que podia consultar o estado do projeto em qualquer momento, sem necessidade de reuniões presenciais constantes.

Para garantir a qualidade do código, a rastreabilidade das alterações e a consistência das versões, utilizou-se o sistema de controlo de versões Git, com recurso à interface gráfica GitHub Desktop. Esta escolha deveu-se à simplicidade de utilização e à integração com o GitHub, que permitiu manter um repositório remoto seguro e acessível. Foram adotadas boas práticas de gestão de repositório, como a criação de commits descritivos (com mensagens

que explicam o propósito das alterações) e a manutenção de uma estrutura clara de branches, separando o desenvolvimento principal (branch ‘main’) de eventuais experiências ou correções urgentes (branches ‘feature/*’ e ‘hotfix/*’). Esta prática facilitou a colaboração (embora o projeto tenha sido individual) e assegurou um histórico completo e organizado de todas as modificações realizadas ao longo do estágio, servindo também como documentação implícita do processo de desenvolvimento. A possibilidade de reverter alterações ou de comparar versões anteriores revelou-se particularmente útil durante a fase de testes, quando foram detetados erros introduzidos em iterações anteriores.

A metodologia ágil, aliada ao controlo de versões rigoroso, permitiu gerir o projeto de forma eficiente, garantindo que as funcionalidades mais críticas fossem entregues primeiro, que o código se mantivesse estável e que a equipa de intervenção social pudesse experimentar a plataforma em fases precoces, fornecendo orientação para as iterações seguintes. Esta dinâmica de desenvolvimento e validação contínua resultou numa aplicação final mais alinhada com as necessidades reais da entidade de acolhimento, reduzindo o risco de desvios em relação aos requisitos iniciais e aumentando a satisfação dos utilizadores finais.

3.2 Tecnologias de Desenvolvimento

3.2.1 Backend Web

O backend da plataforma constitui o núcleo lógico da aplicação, sendo responsável pelo processamento de todos os pedidos dos utilizadores, pela aplicação das regras de negócio, pelo acesso e gestão dos dados, pela autenticação e autorização, bem como pela geração dinâmica de respostas (páginas HTML, redireccionamentos ou mensagens de erro). Esta camada foi desenvolvida na linguagem Python com recurso ao framework Django (versão 6.0), complementado por SQL puro para consultas analíticas complexas e pelo sistema de gestão de base de dados PostgreSQL (versão 16). A escolha do Django deveu-se à sua simplicidade, ao conjunto robusto de ferramentas integradas (como o ORM, o sistema de rotas URL e a camada de autenticação) e à vasta documentação disponível, fatores que aceleraram o desenvolvimento e garantiram a segurança da aplicação.

O backend segue o padrão MTV (Model-Template-View) do Django, mas com uma particularidade relevante: os modelos declarados no ficheiro `models.py` possuem a opção `managed = False`, indicando que o Django não deve gerir migrações. Esta abordagem foi adoptada porque a base de dados foi criada manualmente através de scripts SQL, respeitando as boas práticas de normalização (terceira forma normal) e integridade referencial, e nunca

sofre alterações estruturais sem intervenção explícita. Desta forma, o ORM do Django é utilizado principalmente para consultas simples e para operações de escrita em tabelas com chave primária simples (ex: `UTENTES`, `INTERVENCAO`), enquanto as consultas mais complexas são escritas em SQL puro, garantindo controlo total sobre a performance e a semântica. Exemplos de situações em que se optou pelo SQL puro incluem a geração do dashboard de indicadores (que recorre a vistas como `vw_evolucao_problematicas` e `vw_taxa_reincidencia_por_problematika`), a extração de dados dos snapshots JSONB armazenados nas tabelas de histórico e a execução de atualizações condicionais durante a edição de utentes (como a definição de `data_fim` e `id_intervencao` na tabela `TER`).

Para garantir a consistência dos dados em operações que envolvem múltiplas tabelas – criação de um utente (inserção em `UTENTES`, `UTENTES_SOCIO_ECONOMICAS`, membros do agregado, despesas, escalões e problemáticas), edição de um utente (atualização de vários registos e resolução de problemáticas com datas) e gestão de intervenções – todas estas operações foram envolvidas em blocos `with transaction.atomic()`. Esta abordagem assegura que, se alguma operação falhar (por exemplo, uma violação de chave estrangeira ou um erro de validação), todas as alterações são revertidas, mantendo a integridade da base de dados.

O sistema de autenticação baseia-se no modelo User padrão do Django, complementado por uma tabela `ADMINISTRADORES` que guarda o cargo e outros dados relevantes. A view `login_view` autentica o utilizador através do email e palavra-passe (utilizando o Django Auth) e verifica adicionalmente se o email existe na tabela `ADMINISTRADORES`. Todas as restantes views são protegidas com o decorador `@login_required`, e a proteção CSRF está ativa em todos os formulários submetidos por POST. Os parâmetros SQL utilizam sempre placeholders (`%s`) para evitar injeção de SQL.

A integração com o frontend é feita através do motor de templates do Django, que renderiza as páginas HTML no servidor. Os dados dinâmicos são passados no contexto, e para os gráficos do dashboard, são serializados em JSON com a tag `json_script`, acessíveis posteriormente pelo JavaScript (Chart.js). A paginação das listagens é implementada com o Paginator do Django, e os parâmetros de filtro (ex: pesquisa por nome) são mantidos nos links de navegação para preservar o estado da consulta.

Por fim, o backend recorre ao PostgreSQL não apenas pela sua fiabilidade e desempenho, mas também pelo suporte a tipos de dados avançados como o JSONB, utilizado para armazenar os snapshots de histórico, e pela capacidade de criar vistas materializadas (embora não tenham sido necessárias devido ao volume moderado de dados). Esta

combinação de Django, SQL puro e PostgreSQL revelou-se adequada às necessidades do projeto, proporcionando um equilíbrio entre produtividade no desenvolvimento e eficiência nas consultas analíticas.

3.2.2 Base de Dados

O esquema relacional foi desenhado na terceira forma normal, garantindo a eliminação de redundâncias e a preservação da integridade dos dados. Abrange as principais entidades do domínio da intervenção social comunitária: utentes (diferenciados entre socioeconómicos e idosos), intervenções, encaminhamentos, contactos, problemáticas, agregados familiares, despesas fixas, escalões de rendimento, quedas, atividades de vida diária, doenças crónicas, barreiras arquitetónicas, sinais de fragilidade, administradores e tabelas de domínio (como estado civil, grau de autonomia, nível de isolamento, entre outras). Para assegurar a integridade referencial, foram estabelecidas relações de chave estrangeira com a restrição ‘ON DELETE RESTRICT’, evitando a remoção acidental de registos com dependências activas.

De modo a suportar as análises estatísticas e o dashboard, foram construídas várias vistas SQL (normais), que agregam indicadores estratégicos, tais como: intervenções e contactos por mês, evolução temporal das problemáticas, taxa de reincidência por problemática, idosos em situação de isolamento elevado e a viver sozinhos, quedas repetidas nos últimos seis meses, falta de rede de suporte (idosos que vivem sozinhos e não dispõem de contactos de emergência nem de familiar de referência), visitas prioritárias (com base na conjugação de vários critérios de risco), distribuição etária (para socioeconómicos e idosos), distribuição por género, níveis de autonomia e de isolamento, risco social e utentes por zona de residência. Estas vistas são utilizadas exclusivamente para leitura, fornecendo dados pré-calculados ao dashboard e aos relatórios, o que reduz a carga computacional em cada pedido e garante tempos de resposta adequados.

3.2.3 Frontend

O frontend da plataforma, responsável pela interface com o utilizador, foi implementado com recurso às tecnologias web fundamentais – HTML5, CSS3 e JavaScript – e fez uso intensivo da framework Bootstrap para a estilização das páginas, garantindo uma aparência moderna, consistente e responsiva em diferentes dispositivos e resoluções de ecrã. Os ícones utilizados nos botões, cabeçalhos e menus foram fornecidos pela biblioteca Bootstrap Icons, contribuindo para uma comunicação visual clara e intuitiva. As views do Django devolvem

templates HTML renderizados no servidor, tirando partido do motor de templates integrado, que permite a inclusão de lógica simples (ciclos, condições, formatação de datas) e a reutilização de componentes comuns (como a barra de navegação e o rodapé) através de herança de templates. Os dados dinâmicos são passados no contexto, sendo posteriormente interpolados no HTML ou utilizados para pré-preencher formulários. Para os gráficos do dashboard, os dados agregados (ex: evolução mensal de intervenções, distribuição etária, taxas de reincidência) são serializados em JSON utilizando a tag “`json_script`”, o que garante a segurança contra injeção de código malicioso e a correta transferência de estruturas de dados complexas para o JavaScript. No lado do cliente, a biblioteca Chart.js consome esses dados e desenha gráficos de linhas, barras e pizza, permitindo uma visualização clara e interativa dos indicadores de gestão. A paginação das listagens (utentes, idosos, intervenções, encaminhamentos, contactos) é implementada no servidor com o “Paginator” do Django, e os parâmetros de filtro (por exemplo, a pesquisa por nome) são mantidos nos links de navegação entre páginas, preservando assim o contexto da consulta. Para campos com muitas opções (como a seleção de uma intervenção para associar a um encaminhamento, ou a escolha de um idoso para registar um contacto), foram utilizadas bibliotecas como Select2, que transformam os elementos “`<select>`” em campos pesquisáveis com autocomplete, melhorando significativamente a experiência do utilizador quando as listas de opções são extensas. Em formulários dinâmicos (membros do agregado, despesas fixas, escalões de rendimento, quedas), recorreu-se a JavaScript para adicionar ou remover linhas de tabela, evitando a submissão de dados vazios e proporcionando flexibilidade no preenchimento. O resultado é uma interface funcional, acessível e alinhada com as necessidades da equipa de intervenção social, que pode registar e consultar informação de forma rápida e eficiente.

3.2.4 Geração de Relatórios

A geração de relatórios em formato PDF constitui uma funcionalidade essencial da plataforma, permitindo à equipa de intervenção social obter documentos estruturados e prontos a arquivar ou a partilhar com entidades parceiras e financiadoras. Esta componente foi implementada com recurso à biblioteca “ReportLab”, uma ferramenta Python que oferece um conjunto abrangente de primitivas para a construção programática de documentos PDF, incluindo parágrafos estilizados, tabelas com grelhas, quebras de página, cabeçalhos, rodapés e formatação avançada (cores, alinhamentos e tipos de letra). Optou-se pelo ReportLab devido à sua maturidade, à boa integração com o Django e à capacidade de gerar documentos complexos sem depender de software externo.

Foram desenvolvidos dois tipos principais de relatórios: o “relatório mensal” (que detalha as atividades e indicadores de um determinado mês) e o “relatório anual” (que sintetiza os dados de todo o ano civil). Ambos os relatórios são gerados com base no período temporal selecionado pelo utilizador, respeitando rigorosamente as datas de início e fim (ex: primeiro e último dia do mês ou do ano). Para garantir a exatidão histórica, todas as consultas SQL que alimentam os relatórios utilizam parâmetros de filtragem temporal – sejam as datas de intervenções, contactos, problemáticas, encaminhamentos ou os snapshots das tabelas de histórico. Desta forma, um relatório referente a um ano passado reflete fielmente o estado dos utentes e dos eventos naquela época, e não a situação atual.

A construção de cada relatório envolve a execução de várias consultas SQL (em grande parte baseadas em vistas predefinidas) e a posterior inserção dos dados em tabelas e parágrafos no documento PDF. As principais secções incluem: indicadores gerais (totais de utentes criados no período, intervenções e contactos ocorridos), problemáticas mais frequentes, evolução de problemáticas, taxa de reincidência, tipos de intervenção e programas mais utilizados, respostas de encaminhamentos, perfis etários e de género, níveis de autonomia e isolamento dos idosos, idosos em situação de isolamento elevado, idosos com mais contactos, quedas repetidas, falta de rede de suporte, visitas prioritárias e distribuição de utentes por zona de risco social.

Todas as consultas utilizam placeholders (“%s”) e são parametrizadas com as datas de início e fim do período, garantindo a proteção contra injeção de SQL e a reutilização da mesma lógica para diferentes intervalos. Além disso, para os indicadores que dependem do estado dos utentes (ex: distribuição etária, género, autonomia), os relatórios recorrem às tabelas de histórico (“historico_utente_socioeconomico” e “historico_idoso”) e selecionam o snapshot mais recente até à data de fim do período, assegurando a coerência histórica. A geração dos PDF é acionada através de um formulário no dashboard, onde o utilizador introduz o ano (obrigatório) e opcionalmente o mês. Se o mês for omitido, é gerado o relatório anual; caso contrário, o relatório mensal. O ficheiro resultante é devolvido ao navegador para download, sendo o nome do ficheiro automaticamente composto com o período selecionado (ex: “relatorio_mensal_2025_03.pdf”). Os relatórios assim produzidos servem de suporte à decisão administrativa, fundamentando pedidos de financiamento, a alocação de recursos humanos e a elaboração de planos de intervenção baseados em dados concretos.

3.2.5 Ferramentas Utilizadas

Ao longo do desenvolvimento do projeto, recorreu-se a um conjunto diversificado de ferramentas que apoiaram as diferentes fases do ciclo de vida do software – desde a codificação e gestão de versões até à pesquisa e resolução de problemas.

Como editor de código principal, foi utilizado o “Visual Studio Code”, um ambiente de desenvolvimento integrado (IDE) leve, altamente extensível e com suporte nativo para Python, Django, HTML, CSS, JavaScript e SQL. A integração com Git, contribuiu, também, para a produtividade e para a manutenção de um código limpo e consistente.

Para o controlo de versões e a gestão do histórico do código, recorreu-se ao “Git”, utilizando a interface gráfica “GitHub Desktop” e o repositório remoto alojado no “GitHub”. Esta abordagem permitiu manter um registo detalhado de todas as alterações, facilitar a experimentação de novas funcionalidades em “branches” separadas e assegurar a recuperação do código em caso de erro. O GitHub serviu ainda como plataforma de backup e de partilha (embora o projeto tenha sido individual).

Em situações pontuais de dúvida sobre sintaxe, conceitos avançados ou depuração de erros, foram utilizadas ferramentas de inteligência artificial, nomeadamente o “ChatGPT” e o “DeepSeek”. Estas ferramentas auxiliaram na geração de explicações alternativas, na sugestão de correções para mensagens de erro e na proposta de abordagens para consultas SQL complexas ou para a estruturação de vistas. Sempre que as soluções sugeridas foram aproveitadas, foram analisadas criticamente, adaptadas ao contexto do projeto e integradas após validação e compreensão do seu funcionamento. Adicionalmente, o motor de busca “Google” foi utilizado frequentemente para encontrar documentação oficial, tutoriais e respostas em fóruns especializados (como o Stack Overflow), complementando as fontes primárias de informação.

4 Atividades desenvolvidas

Neste capítulo descrevem-se, de forma sucinta e clara, as tarefas realizadas ao longo do estágio, desde a fase inicial de análise de requisitos até à implementação e testes da plataforma web. São também justificadas as principais decisões tomadas ao longo do

processo, com base nas necessidades da entidade de acolhimento e nas boas práticas de engenharia de software.

4.1 Análise de Requisitos

A fase de análise de requisitos constituiu o ponto de partida do projeto, tendo como objetivo compreender em profundidade as necessidades da entidade de acolhimento, bem como definir as funcionalidades e as características que a plataforma deveria possuir. Esta etapa envolveu reuniões com os técnicos de intervenção social da Junta de Freguesia de Viseu, a observação dos processos manuais em uso (registos em papel, folhas de cálculo e arquivos físicos) e a identificação dos principais problemas operacionais, nomeadamente a dispersão da informação, a dificuldade em gerar indicadores de gestão e a morosidade na consulta de históricos de utentes.

Foram assim levantados os requisitos funcionais, ou seja, as ações concretas que a plataforma deve suportar. Entre eles, destacam-se a gestão de utentes socioeconómicos e idosos (registo, consulta, edição e eliminação), a gestão de intervenções associadas a utentes socioeconómicos, a gestão de encaminhamentos para entidades parceiras, a gestão de contactos com idosos, a geração de relatórios anuais e mensais em formato PDF, a apresentação de um dashboard com indicadores estatísticos e a manutenção de um histórico de alterações de cada utente (rastreadibilidade).

Para além dos requisitos funcionais, foram definidos requisitos não funcionais, os quais especificam critérios de qualidade e restrições do sistema. Entre eles, incluem-se a usabilidade (interface intuitiva e adaptável a diferentes níveis de literacia digital), o desempenho (tempos de resposta adequados mesmo com um número elevado de registos), a segurança (controlo de acessos por autenticação, proteção contra injeção de SQL, conformidade com o RGPD), a disponibilidade (acesso contínuo durante o horário de funcionamento da junta) e a escalabilidade (capacidade de evolução futura sem comprometer a estrutura existente). Estes requisitos orientaram a escolha das tecnologias (Django, PostgreSQL, Bootstrap) e a própria arquitetura da aplicação.

A análise detalhada dos requisitos foi documentada e validada pela entidade de acolhimento, servindo de base para as fases seguintes de modelação da base de dados, desenvolvimento e testes.

4.2 Modelação da Base de Dados

[illegible]

20

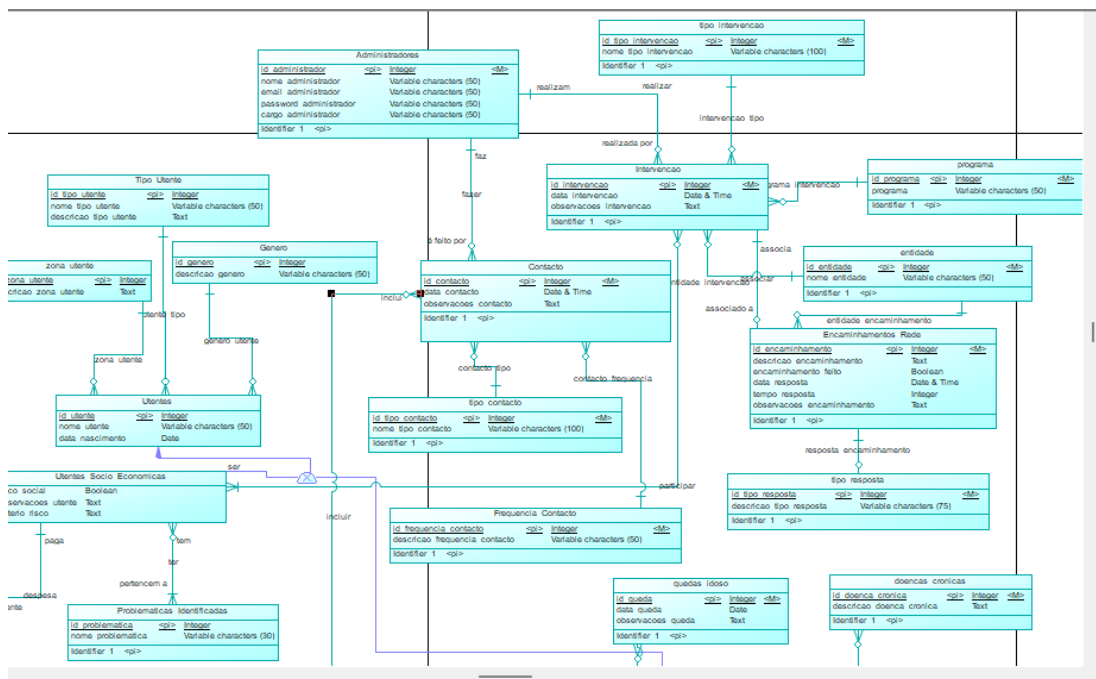


Figura 2 - Segunda Parte do Modelo Conceptual de Dados (PowerDesigner)

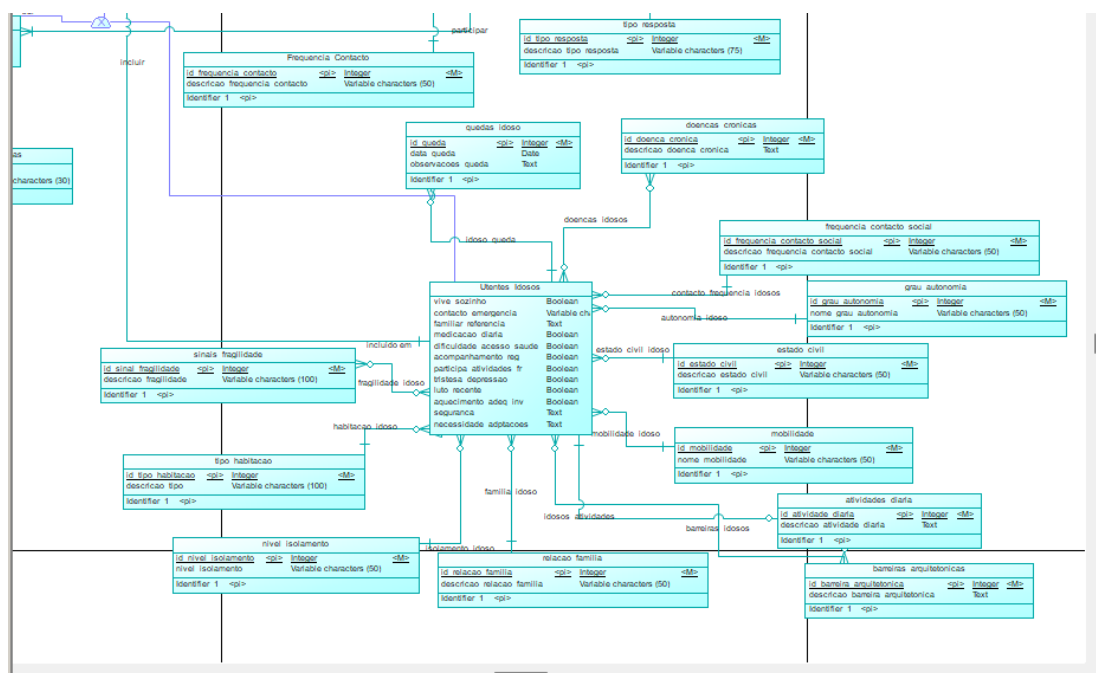


Figura 3 - Última Parte do Modelo Conceptual de Dados (PowerDesigner)

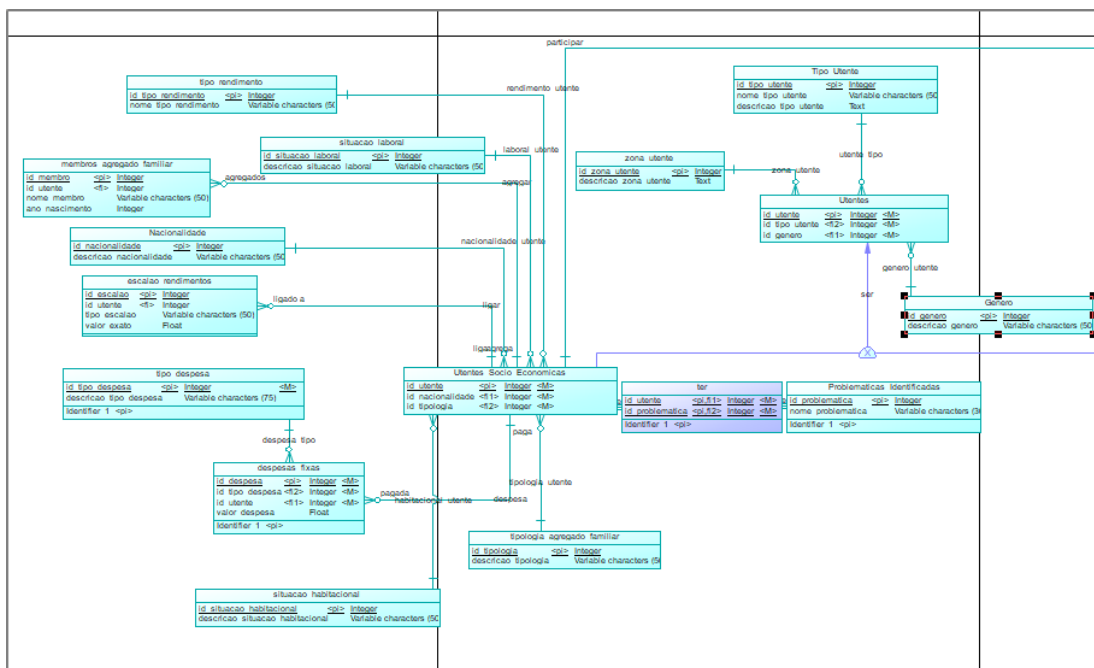


Figura 4 - Primeira Parte do Modelo Lógico de Dados (PowerDesigner)

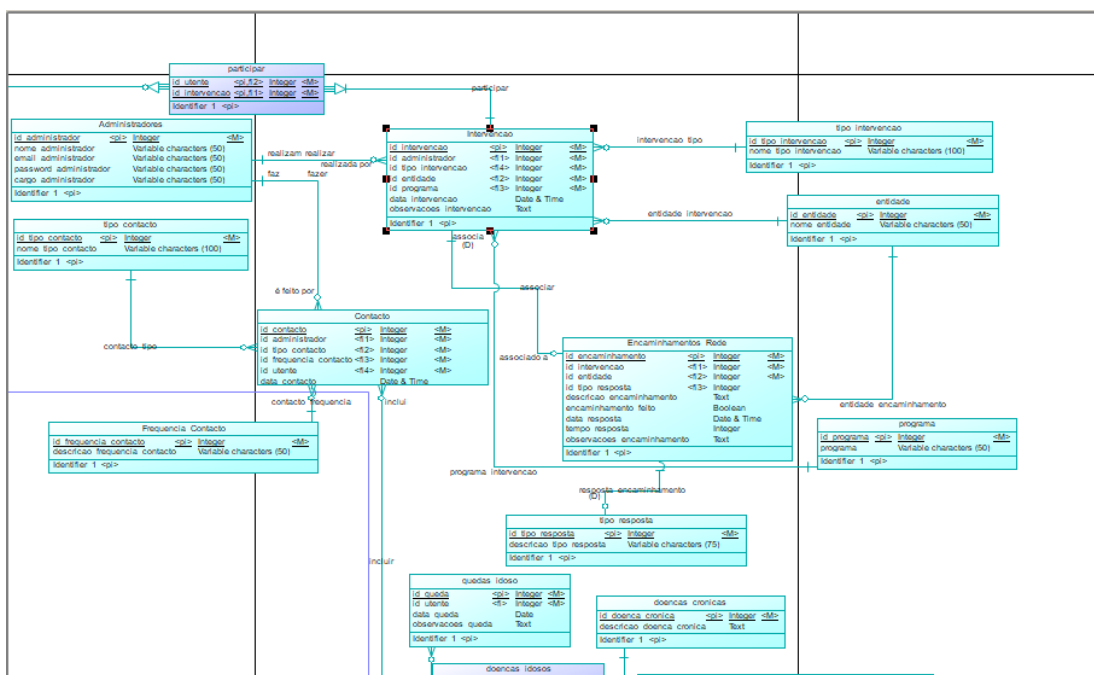
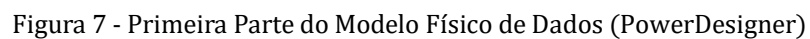


Figura 5 - Segunda Parte do Modelo Lógico de Dados (PowerDesigner)



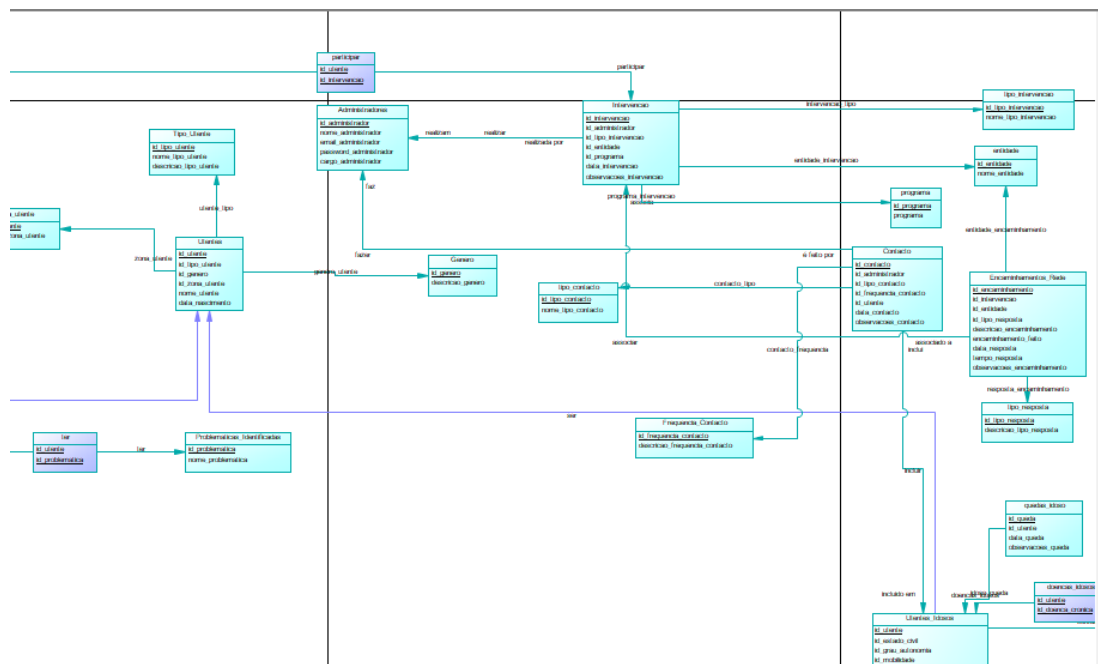


Figura 8 - Segunda Parte do Modelo Físico de Dados (PowerDesigner)

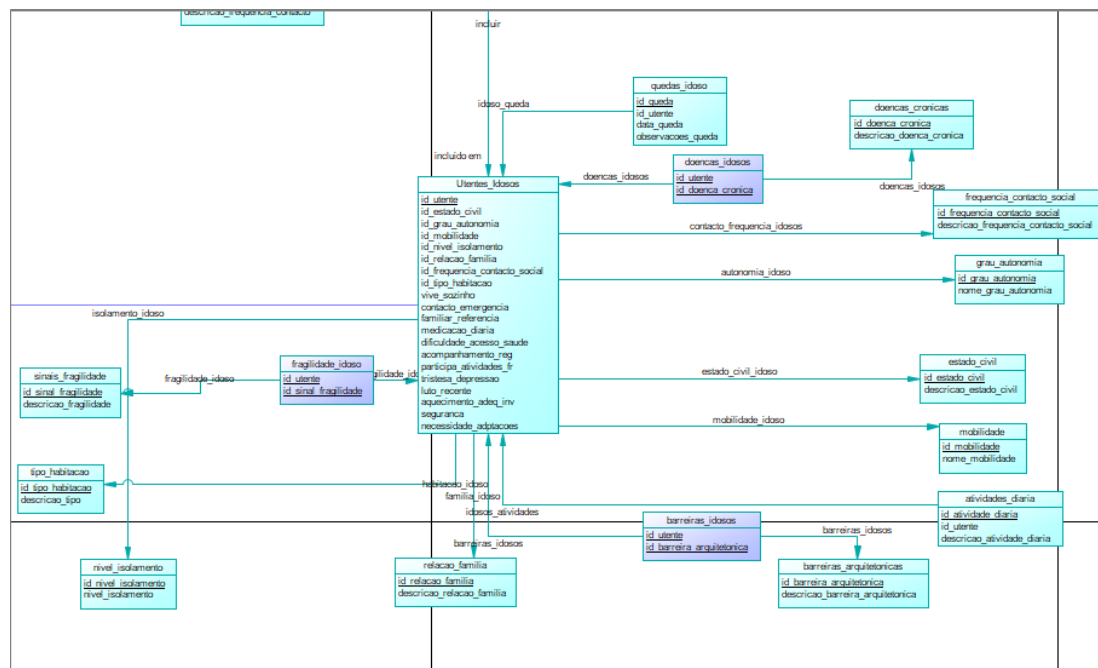


Figura 9 - Última Parte do Modelo Físico de Dados (PowerDesigner)

```

/*=====*/
/* DBMS name:      PostgreSQL 14.x */
/* Created on:     24/03/2026 17:27:49 */
/*=====*/
/*=====*/
/* DROPS ORDENADOS + IF EXISTS */
/*=====*/
DROP TABLE IF EXISTS historico_idoso;
DROP TABLE IF EXISTS HISTORICO_UTENTE_SOCIOECONOMICO;
DROP TABLE IF EXISTS TER;
DROP TABLE IF EXISTS PARTICIPAR;
DROP TABLE IF EXISTS ENCAMINHAMENTOS_REDE;
DROP TABLE IF EXISTS CONTACTO;
DROP TABLE IF EXISTS INTERVENCAO;
DROP TABLE IF EXISTS ATIVIDADES_DIARIA;
DROP TABLE IF EXISTS BARREIRAS_IDOSOS;
DROP TABLE IF EXISTS DOENCAS_IDOSOS;
DROP TABLE IF EXISTS FRAGILIDADE_IDOSO;
DROP TABLE IF EXISTS QUEDAS_IDOSO;
DROP TABLE IF EXISTS DESPESAS_FIXAS;
DROP TABLE IF EXISTS ESCALAO_RENDIMENTOS;
DROP TABLE IF EXISTS MEMBROS_AGREGADO_FAMILIAR;
DROP TABLE IF EXISTS UTENTES_IDOSOS;
DROP TABLE IF EXISTS UTENTES_SOCIO_ECONOMICAS;
DROP TABLE IF EXISTS UTENTES;
DROP TABLE IF EXISTS ADMINISTRADORES;
DROP TABLE IF EXISTS TIPO_CONTACTO;
DROP TABLE IF EXISTS FREQUENCIA_CONTACTO;
DROP TABLE IF EXISTS TIPO_INTERVENCAO;
DROP TABLE IF EXISTS ENTIDADE;
DROP TABLE IF EXISTS PROGRAMA;
DROP TABLE IF EXISTS TIPO_RESPOSTA;

```

Figura 10 - Secção dos DROP TABLES do script de criação de tabelas (DDL)

```

/*=====*/
create table ADMINISTRADORES (
  ID_ADMINISTRADOR      bigserial          not null,
  NOME_ADMINISTRADOR    varchar(50),
  EMAIL_ADMINISTRADOR   varchar(50),
  PASSWORD_ADMINISTRADOR varchar(50),
  CARGO_ADMINISTRADOR   varchar(50),
  constraint PK_ADMINISTRADORES primary key (ID_ADMINISTRADOR)
);

/*=====*/
/* Table: ESTADO_CIVIL */
/*=====*/
create table ESTADO_CIVIL (
  ID_ESTADO_CIVIL      bigserial          not null,
  DESCRICAO_ESTADO_CIVIL varchar(50),
  constraint PK_ESTADO_CIVIL primary key (ID_ESTADO_CIVIL)
);

/*=====*/
/* Table: GRAU_AUTONOMIA */
/*=====*/
create table GRAU_AUTONOMIA (
  ID_GRAU_AUTONOMIA      bigserial          not null,
  NOME_GRAU_AUTONOMIA    varchar(50),
  constraint PK_GRAU_AUTONOMIA primary key (ID_GRAU_AUTONOMIA)
);

/*=====*/
/* Table: MOBILIDADE */
/*=====*/
create table MOBILIDADE (

```

Figura 11 - Criação de tabelas da base de dados

```

/*=====*/
/* Table: MOBILIDADE */
/*=====*/
create table MOBILIDADE (
  ID_MOBILIDADE      bigserial          not null,
  NOME_MOBILIDADE    varchar(50),
  constraint PK_MOBILIDADE primary key (ID_MOBILIDADE)
);

/*=====*/
/* Table: NIVEL_ISOLAMENTO */
/*=====*/
create table NIVEL_ISOLAMENTO (
  ID_NIVEL_ISOLAMENTO bigserial          not null,
  NIVEL_ISOLAMENTO    varchar(50),
  constraint PK_NIVEL_ISOLAMENTO primary key (ID_NIVEL_ISOLAMENTO)
);

/*=====*/
/* Table: RELACAO_FAMILIA */
/*=====*/
create table RELACAO_FAMILIA (
  ID_RELACAO_FAMILIA bigserial          not null,
  DESCRICAO_RELACAO_FAMILIA varchar(50),
  constraint PK_RELACAO_FAMILIA primary key (ID_RELACAO_FAMILIA)
);

/*=====*/
/* Table: FREQUENCIA_CONTACTO_SOCIAL */
/*=====*/
create table FREQUENCIA_CONTACTO_SOCIAL (

```

Figura 12 - Criação de Tabelas Auxiliares

```

/*=====*/
/* Table: UTENTES */
/*=====*/
create table UTENTES (
  ID_UTENTE          bigserial          not null,
  ID_TIPO_UTENTE     bigint             not null,
  ID_GENERO          bigint             not null,
  ID_ZONA_UTENTE     bigint             not null,
  NOME_UTENTE        varchar(50),
  DATA_NASCIMENTO   date,
  DATA_CRIACAO      date DEFAULT CURRENT_DATE,
  constraint PK_UTENTES primary key (ID_UTENTE),
  constraint FK_UTENTES_UTENTE_TI_TIPO_UTE foreign key (ID_TIPO_UTENTE)
    references TIPO_UTENTE (ID_TIPO_UTENTE)
    on delete restrict on update restrict,
  constraint FK_UTENTES_GENERO_UT_GENERO foreign key (ID_GENERO)
    references GENERO (ID_GENERO)
    on delete restrict on update restrict,
  constraint FK_UTENTES_ZONA_UTEN_ZONA_UTE foreign key (ID_ZONA_UTENTE)
    references ZONA_UTENTE (ID_ZONA_UTENTE)
    on delete restrict on update restrict
);

/*=====*/
/* Table: UTENTES_IDOSOS */
/*=====*/
create table UTENTES_IDOSOS (
  ID_UTENTE          bigint             not null,
  ID_ESTADO_CIVIL    bigint             not null,
  ID_GRAU_AUTONOMIA  bigint             not null,
  ID_MOBILIDADE      bigint             not null,

```

Figura 13 - Criação das Tabelas Utentes e Utentes Idosos

```

/*=====*/
/* Table: CONTACTO */
/*=====*/
create table CONTACTO (
  ID_CONTACTO          bigint          not null,
  ID_ADMINISTRADOR     bigint          not null,
  ID_TIPO_CONTACTO     bigint          not null,
  ID_FREQUENCIA_CONTACTO bigint        not null,
  ID_UTENTE            bigint          not null,
  DATA_CONTACTO       timestamp,
  OBSERVACOES_CONTACTO text,
  constraint PK_CONTACTO primary key (ID_CONTACTO),
  constraint FK_CONTACTO_FAZER_ADMINIST foreign key (ID_ADMINISTRADOR)
    references ADMINISTRADORES (ID_ADMINISTRADOR)
    on delete restrict on update restrict,
  constraint FK_CONTACTO_INCLUIR_UTENTES_ foreign key (ID_UTENTE)
    references UTENTES_IDOSOS (ID_UTENTE)
    on delete restrict on update restrict,
  constraint FK_CONTACTO_CONTACTO_TIPO_CON foreign key (ID_TIPO_CONTACTO)
    references TIPO_CONTACTO (ID_TIPO_CONTACTO)
    on delete restrict on update restrict,
  constraint FK_CONTACTO_CONTACTO_FREQUENC foreign key (ID_FREQUENCIA_CONTACTO)
    references FREQUENCIA_CONTACTO (ID_FREQUENCIA_CONTACTO)
    on delete restrict on update restrict
);

```

Figura 14 - Criação da Tabela Contactos de Idosos

```

/*=====*/
/* Table: UTENTES_SOCIO_ECONOMICAS */
/*=====*/
create table UTENTES_SOCIO_ECONOMICAS (
  ID_UTENTE            bigint          not null,
  ID_NACIONALIDADE     bigint          not null,
  ID_TIPOLOGIA         bigint          not null,
  ID_SITUACAO_HABITACIONAL bigint        not null,
  ID_SITUACAO_LABORAL  bigint          not null,
  ID_TIPO_RENDIMENTO   bigint          not null,
  RISCO_SOCIAL         boolean,
  OBSERVACOES_UTENTE   text,
  CRITERIO_RISCO       text,
  constraint PK_UTENTES_SOCIO_ECONOMICAS primary key (ID_UTENTE),
  constraint FK_UTENTES_NACIONALI_NACIONAL foreign key (ID_NACIONALIDADE)
    references NACIONALIDADE (ID_NACIONALIDADE)
    on delete restrict on update restrict,
  constraint FK_UTENTES_TIPOLOGIA_TIPOLOGI foreign key (ID_TIPOLOGIA)
    references TIPOLOGIA_AGREGADO_FAMILIAR (ID_TIPOLOGIA)
    on delete restrict on update restrict,
  constraint FK_UTENTES_HABITACIO_SITUACAO foreign key (ID_SITUACAO_HABITACIONAL)
    references SITUACAO_HABITACIONAL (ID_SITUACAO_HABITACIONAL)
    on delete restrict on update restrict,
  constraint FK_UTENTES_LABORAL_U_SITUACAO foreign key (ID_SITUACAO_LABORAL)
    references SITUACAO_LABORAL (ID_SITUACAO_LABORAL)
    on delete restrict on update restrict,
  constraint FK_UTENTES_RENDIMENTO_TIPO_REN foreign key (ID_TIPO_RENDIMENTO)
    references TIPO_RENDIMENTO (ID_TIPO_RENDIMENTO)
    on delete restrict on update restrict,
  constraint FK_UTENTES_SER_UTENTES foreign key (ID_UTENTE)
    references UTENTES (ID_UTENTE)
    on delete restrict on update restrict
);

```

Figura 15 - Criação da Tabela Utentes Socioeconómicos

```

/*-----*/
/* Table: INTERVENCAO */
/*-----*/
create table INTERVENCAO (
  ID_INTERVENCAO      bigserial          not null,
  ID_ADMINISTRADOR    bigint             not null,
  ID_TIPO_INTERVENCAO bigint             not null,
  ID_ENTIDADE         bigint             not null,
  ID_PROGRAMA         bigint             not null,
  DATA_INTERVENCAO   timestamp,
  OBSERVACOES_INTERVENCAO text,
  constraint PK_INTERVENCAO primary key (ID_INTERVENCAO),
  constraint FK_INTERVEN_REALIZAR_ADMINIST foreign key (ID_ADMINISTRADOR)
    references ADMINISTRADORES (ID_ADMINISTRADOR)
    on delete restrict on update restrict,
  constraint FK_INTERVEN_INTERVENC_TIPO_INT foreign key (ID_TIPO_INTERVENCAO)
    references TIPO_INTERVENCAO (ID_TIPO_INTERVENCAO)
    on delete restrict on update restrict,
  constraint FK_INTERVEN_ENTIDADE_ENTIDADE foreign key (ID_ENTIDADE)
    references ENTIDADE (ID_ENTIDADE)
    on delete restrict on update restrict,
  constraint FK_INTERVEN_PROGRAMA_PROGRAMA foreign key (ID_PROGRAMA)
    references PROGRAMA (ID_PROGRAMA)
    on delete restrict on update restrict
);

```

Figura 16 - Criação da Tabela Intervenção

```

/*-----*/
/* Table: ENCAMINHAMENTOS_REDE */
/*-----*/
create table ENCAMINHAMENTOS_REDE (
  ID_ENCAMINHAMENTO    bigserial          not null,
  ID_INTERVENCAO       bigint             not null,
  ID_ENTIDADE          bigint             not null,
  ID_TIPO_RESPOSTA     bigint,
  DESCRICAO_ENCAMINHAMENTO text,
  ENCAMINHAMENTO_FEITO boolean,
  DATA_RESPOSTA       timestamp,
  TEMPO_RESPOSTA       integer,
  OBSERVACOES_ENCAMINHAMENTO text,
  constraint PK_ENCAMINHAMENTOS_REDE primary key (ID_ENCAMINHAMENTO),
  constraint FK_ENCAMINH_ASSOCIAR_INTERVEN foreign key (ID_INTERVENCAO)
    references INTERVENCAO (ID_INTERVENCAO)
    on delete restrict on update restrict,
  constraint FK_ENCAMINH_ENTIDADE_ENTIDADE foreign key (ID_ENTIDADE)
    references ENTIDADE (ID_ENTIDADE)
    on delete restrict on update restrict,
  constraint FK_ENCAMINH_RESPOSTA_TIPO_RES foreign key (ID_TIPO_RESPOSTA)
    references TIPO_RESPOSTA (ID_TIPO_RESPOSTA)
    on delete restrict on update restrict
);

```

Figura 17 - Criação da Tabela Encaminhamentos

```

create table TER (
  ID_TER          bigserial          not null,
  ID_UTENTE       bigint             not null,
  ID_PROBLEMATICA bigint             not null,
  DATA_INICIO    date               not null,
  DATA_FIM       date,
  ID_INTERVENCAO  bigint,
  constraint PK_TER primary key (ID_TER),
  constraint FK_TER_TER2_UTENTES foreign key (ID_UTENTE)
    references UTENTES_SOCIO_ECONOMICAS (ID_UTENTE)
    on delete restrict on update restrict,
  constraint FK_TER_TER_PROBLEMA foreign key (ID_PROBLEMATICA)
    references PROBLEMAS_IDENTIFICADAS (ID_PROBLEMATICA)
    on delete restrict on update restrict,
  CONSTRAINT fk_ter_intervencao FOREIGN KEY (id_intervencao) REFERENCES intervencao(id_intervencao) ON DELETE SET NULL
);
CREATE TABLE HISTORICO_UTENTE_SOCIOECONOMICO (
  ID_HISTORICO    BIGSERIAL PRIMARY KEY,
  ID_UTENTE       BIGINT NOT NULL,
  DATA_HISTORICO TIMESTAMP NOT NULL DEFAULT NOW(),
  USUARIO         VARCHAR(100),
  SNAPSHOT_JSON   JSONB NOT NULL,
  CONSTRAINT fk_hist_socio_utente FOREIGN KEY (ID_UTENTE) REFERENCES UTENTES (ID_UTENTE) ON DELETE RESTRICT ON UPDATE RESTRICT;
);
CREATE TABLE IF NOT EXISTS historico_idoso (
  id_historico BIGSERIAL PRIMARY KEY,
  id_utente    BIGINT NOT NULL,
  data_historico TIMESTAMP NOT NULL DEFAULT NOW(),
  usuario     VARCHAR(100),
  snapshot_json JSONB NOT NULL,
  CONSTRAINT fk_hist_idoso_utente FOREIGN KEY (ID_UTENTE) REFERENCES UTENTES (ID_UTENTE) ON DELETE RESTRICT ON UPDATE RESTRICT;
);

```

Figura 18 - Criação de Outras Tabelas

4.3 Implementação do Backend

A seguinte secção descreve a implementação da camada servidor da plataforma web, responsável pela lógica de negócio, pelo acesso aos dados, pela autenticação de utilizadores e por outras funcionalidades fundamentais da aplicação. Nesta camada, integram-se os modelos de dados desenvolvidos em Python, que representam a estrutura da base de dados criada em SQL e implementada no PostgreSQL, bem como as views que processam os pedidos HTTP dos utilizadores e devolvem as respostas adequadas - páginas HTML com dados dinâmicos, redireccionamentos (endpoints) ou mensagens de erro, consoante o caso.

O backend foi desenvolvido na linguagem Python com recurso ao framework Django. A escolha do Django deveu-se à sua simplicidade e ao conjunto robusto de ferramentas integradas que disponibiliza, nomeadamente o ORM (Object-Relational Mapper), um sistema flexível de rotas URL e um sistema de autenticação completo. Este conjunto permitiu um rápido desenvolvimento e garantiu a segurança de uma aplicação web robusta, respeitando as boas práticas de engenharia de software.

Para consultas complexas em SQL, designadamente as que alimentam o dashboard de dados e os relatórios estatísticos, optou-se por utilizar SQL puro em vez do ORM. Esta abordagem proporcionou um controlo fino sobre as consultas à base de dados, permitindo otimizações específicas, a utilização de funções de janela, subconsultas e agregações avançadas que seriam mais difíceis de expressar de forma eficiente através do ORM.

Como sistema de gestão de base de dados, foi utilizado o PostgreSQL. Esta escolha baseou-se na experiência prévia do estagiário com este sistema, na sua preferência pessoal e, sobretudo, na sua capacidade comprovada para lidar com consultas analíticas, suporte a tipos de dados avançados (como JSONB) e elevada conformidade com os padrões SQL, características essenciais para a fiabilidade e o desempenho da plataforma. A seguinte secção descreve a implementação da camada servidor da plataforma web, que é responsável pela lógica de negócio, pelo acesso aos dados, pela autenticação de utilizadores e por outras funcionalidades essenciais ao funcionamento da aplicação. Nesta camada, integram-se os modelos de dados desenvolvidos em Python, os quais representam a estrutura da base de dados criada em SQL e implementada no PostgreSQL, bem como as views que processam os pedidos HTTP dos utilizadores, devolvendo as respostas adequadas — páginas HTML com dados dinâmicos, redireccionamentos (endpoints) ou mensagens de erro, consoante o caso específico.

O backend foi desenvolvido na linguagem Python com recurso ao framework Django. A escolha do Django justifica-se pela sua simplicidade e pelo conjunto robusto de ferramentas integradas que disponibiliza, nomeadamente o ORM (Object-Relational Mapper), um sistema flexível de rotas URL e um sistema de autenticação completo. Este conjunto permitiu um rápido desenvolvimento e garantiu a segurança de uma aplicação web robusta, respeitando as boas práticas da engenharia de software.

Para consultas complexas em SQL, designadamente as que alimentam o dashboard de dados e os relatórios estatísticos, optou-se por utilizar SQL puro em vez do ORM. Esta abordagem proporcionou um controlo fino sobre as consultas à base de dados, permitindo otimizações específicas, a utilização de funções de janela, subconsultas e agregações avançadas que seriam mais difíceis de expressar de forma eficiente através do ORM.

Como sistema de gestão de base de dados, foi utilizado o PostgreSQL. Esta escolha baseou-se na experiência prévia do estagiário com este sistema, na sua preferência pessoal e, sobretudo, na sua capacidade comprovada para lidar com consultas analíticas, suporte a tipos de dados avançados (como JSONB) e elevada conformidade com os padrões SQL, características essenciais para a fiabilidade e o desempenho da plataforma.

4.3.1 Arquitetura Geral

A aplicação segue o padrão de arquitectura MTV (Model-Template-View) do Django, mas com as seguintes particularidades:

DEFINIÇÃO	DESCRIÇÃO
MODELS	Definidos com managed=false, para reflectir exactamente a base de dados criada manualmente, no PostgreSQL. Utilizou-se AutoField, ForeignKey, OneToOneField e BooleanField, respeitando a normalização e as chaves estrangeiras.
VIEWS	Implementadas com raw SQL e ORM, para consultas complexas de dados e de registo, edição, listagem e eliminação nas tabelas da base de dados.
TEMPLATES	Renderizados com Django, e usando, por exemplo, a passagem de dados em JSON para os gráficos em Chart.js

Tabela 1 - Arquitetura MTV do Django

4.3.2 Modelação da Base de Dados

A base de dados compreende as seguintes tabelas:

TABELA	DESCRIÇÃO
UTENTES	Dados comuns a todos os utentes (nome, data de nascimento, género, zona, tipo de utente, data de criação).
UTENTES IDOSOS	Dados específicos dos idosos (autonomia, mobilidade, isolamento, entre outros).
UTENTES SOCIOECONÓMICOS	Dados socioeconómicos (tipologia do agregado familiar, situação laboral, redimentos, risco social).
TER	Registo das problemáticas associadas a cada utente.
INTERVENÇÃO	Intervenções realizadas aos utentes socioeconómicos.
PARTICIPAR	Tabela de associação Muitos-para-Muitos entre as tabelas de Utentes Socioeconómicos e Intervenção.
ENCAMINHAMENTOS_REDE	Encaminhamentos feitos e associados às intervenções com resposta, data e tempo.
CONTACTO	Contactos realizados com idosos (tipo de contacto, frequência e observações)

ATIVIDADES_DIARIA, DOENCAS_IDOSO, BARREIRAS_IDOSO, FRAGILIDADE_IDOSO, QUEDAS_IDOSO	Dados sobre o estilo de vida dos idosos.
HISTORICO_UTENTE_SOCIOECONOMICO, HISTORICO_IDOSO	Tabelas de armazenamento de históricos das informações de utentes em snapshots JSONB.

Tabela 2 - Tabelas da Base de Dados SQL

Para a plataforma web, foram necessárias vistas criadas em SQL, de forma a implementar o dashboard web, ou seja, a partir das vistas ou consultas analíticas e complexas em SQL puro nos dados da base de dados implementada, foi possível construir um dashboard ou uma página com gráficos detalhados a explicar os dados analisados. A seguir, mostram-se exemplos de três vistas em SQL:

```
CREATE OR REPLACE VIEW vista_totais_gerais AS
SELECT
  (SELECT COUNT(*) FROM UTENTES WHERE id_tipo_utente = 2) as total_idosos,
  (SELECT COUNT(*) FROM UTENTES WHERE id_tipo_utente = 1) as total_socioeconomicos,
  (SELECT COUNT(*) FROM intervencao) as total_intervencoes,
  (SELECT COUNT(*) FROM contacto) as total_contactos;
```

Figura 19 - Vista em SQL dos números totais dos Utentes, Intervenções e Contactos registados

```
CREATE OR REPLACE VIEW vw_intervencoes_mes_ano AS
SELECT
  EXTRACT(YEAR FROM data_intervencao) AS ano,
  EXTRACT(MONTH FROM data_intervencao) AS mes,
  COUNT(*) AS total
FROM intervencao
GROUP BY ano, mes
ORDER BY ano, mes;
```

Figura 20 - Vista do Total de Intervenções por Mês e Ano

```
CREATE OR REPLACE VIEW vw_idosos_isolados AS
SELECT
    u.id_utente,
    u.nome_utente,
    EXTRACT(YEAR FROM AGE(CURRENT_DATE, u.data_nascimento)) AS idade,
    ni.nivel_isolamento AS nivel_isolamento,
    ui.vive_sozinho
FROM UTENTES_IDOSOS ui
JOIN UTENTES u ON u.id_utente = ui.id_utente
JOIN NIVEL_ISOLAMENTO ni ON ni.id_nivel_isolamento = ui.id_nivel_isolamento
WHERE ui.vive_sozinho = true AND ni.nivel_isolamento = 'Elevado';
```

Figura 21 - Vista dos Idosos com Nível de Isolamento Elevado

4.3.3 Implementação dos modelos Django

Os modelos foram declarados com a regra de `manage=false`, visto que já se tinha implementado a base de dado SQL e não era necessário fazer migrações de modelos escritos em Python no `models.py`. A seguir, temos o exemplo do modelo `Utentes`:

```
class Utentes(models.Model):
    id_utente = models.BigAutoField(primary_key=True) # ← alterado
    id_tipo_utente = models.ForeignKey(TipoUtente, models.PROTECT, db_column='id_tipo_utente')
    id_genero = models.ForeignKey(Genero, models.PROTECT, db_column='id_genero')
    id_zona_utente = models.ForeignKey('ZonaUtente', models.PROTECT, db_column='id_zona_utente')
    nome_utente = models.CharField(max_length=50, blank=True, null=True)
    data_nascimento = models.DateField(blank=True, null=True)
    data_criacao = models.DateField(blank=True, null=True)

    class Meta:
        managed = False
        db_table = 'utentes'
```

Figura 22 - Modelo Utentes em Python (`models.py`)

4.3.4 Views (Lógica de negócio)

As views do Django constituem o coração da lógica de negócio da plataforma, sendo responsáveis por processar os pedidos HTTP, interagir com os modelos (e com SQL puro quando necessário), aplicar as regras de validação e devolver as respostas apropriadas – páginas HTML renderizadas, redireccionamentos, mensagens de erro ou ficheiros PDF. Esta subsecção descreve a implementação das principais famílias de views, justificando as opções de desenho e as soluções adoptadas para garantir consistência, segurança e desempenho. As views do Django implementam as seguintes funcionalidades:

- Autenticação: A view `login_view` implementa o processo de autenticação, utilizando o sistema nativo do Django (`django.contrib.auth`). O utilizador submete o seu email e

palavra-passe; o Django autentica-o contra a tabela `auth_user`. Em seguida, é executada uma consulta SQL adicional à tabela `ADMINISTRADORES` para verificar se o email pertence a um administrador autorizado. Esta dupla verificação permite que apenas os técnicos previamente registados na base de dados possam aceder à plataforma. Após autenticação bem-sucedida, o utilizador é redireccionado para a página inicial (dashboard). Todas as restantes views são protegidas pelo decorador `@login_required`, garantindo que nenhuma funcionalidade fica acessível a utilizadores não autenticados. A view `logout_view` termina a sessão e redirecciona para o ecrã de login. A imagem a seguir representa essas views:

```
from django.shortcuts import render, redirect
from django.contrib import messages
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.db import connection

def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")
        user = authenticate(request, username=username, password=password)
        if user is not None:
            # Verificar se está na tabela ADMINISTRADORES
            with connection.cursor() as cursor:
                cursor.execute("""
                    SELECT ID_ADMINISTRADOR
                    FROM ADMINISTRADORES
                    WHERE EMAIL_ADMINISTRADOR = %s
                """, [user.email])
                admin = cursor.fetchone()
            if admin:
                login(request, user)
                return redirect("home")
            else:
                messages.error(request, "Não é um administrador autorizado.")
        else:
            messages.error(request, "Credenciais inválidas.")
    return render(request, "login.html")

def logout_view(request):
    logout(request)
    return redirect("login")
```

Figura 23 - Views de Login e Logout

- **CRUD de Utentes Socioeconómicos:** O módulo de utentes socioeconómicos representa parte do núcleo da plataforma, visto que concentra informação sensível sobre cada utente com vulnerabilidade socioeconómica e as relações complexas. A implementação segue princípios de atomicidade, validação rigorosa e rastreabilidade. A seguir, descrevem-se as operações principais:
 - **Listagem (socioeconomicos_lista):**

A view implementada de `socioeconomicos_lista` é responsável por listar todos os utentes socioeconomicos numa página html, com o filtro por nome do dado utente. A listagem SQL é feita dinamicamente, usando o operador `ILIKE`, quando o nome é fornecido, e os resultados são paginados com o paginador do Django.

A figura seguinte mostra o código dessa vista em python implementada:

```
@login_required
def socioeconomicos_lista(request):
    nome_filter = request.GET.get('nome', '').strip()
    page_number = request.GET.get('page', 1)
    query = """
    SELECT
        u.ID_UTENTE,
        u.NOME_UTENTE,
        EXTRACT(YEAR FROM AGE(CURRENT_DATE, u.DATA_NASCIMENTO)) AS idade,
        n.DESCRICAO_NACIONALIDADE,
        z.DESCRICAO_ZONA_UTENTE,
        sh.DESCRICAO_SITUACAO_HABITACIONAL,
        sl.DESCRICAO_SITUACAO_LABORAL,
        usec.RISCO_SOCIAL
    FROM UTENTES u
    JOIN UTENTES_SOCIO_ECONOMICAS usec ON u.ID_UTENTE = usec.ID_UTENTE
    JOIN NACIONALIDADE n ON usec.ID_NACIONALIDADE = n.ID_NACIONALIDADE
    JOIN ZONA_UTENTE z ON u.ID_ZONA_UTENTE = z.ID_ZONA_UTENTE
    JOIN SITUACAO_HABITACIONAL sh ON usec.ID_SITUACAO_HABITACIONAL = sh.ID_SITUACAO_HABITACIONAL
    JOIN SITUACAO_LABORAL sl ON usec.ID_SITUACAO_LABORAL = sl.ID_SITUACAO_LABORAL
    """
    params = []
    if nome_filter:
        query += " WHERE u.NOME_UTENTE ILIKE %s"
        params.append(f'%{nome_filter}%')
    query += " ORDER BY u.ID_UTENTE"
    with connection.cursor() as cursor:
        cursor.execute(query, params)
        resultados = cursor.fetchall()
    # Paginação
    paginator = Paginator(resultados, 20) # 20 registos por página
    try:
        page_obj = paginator.page(page_number)
    except PageNotAnInteger:
        page_obj = paginator.page(1)
    except EmptyPage:
        page_obj = paginator.page(paginator.num_pages)
    return render(request, 'socioeconomicos/lista_utentes.html', {'utentes': page_obj, 'filtro_nome': nome_filter,})
```

Figura 24 - Vista de Listagem de Utentes Socioeconómicos

- Detalhe (socioeconomicos_detalle):

A página de detalhe mostra todas de um utente específico, dados pessoais, as intervenções em que participou, as problemáticas, com as datas de início, fim e estado, os membros do agregado familiar, as despesas fixas e os escalões de rendimento. As problemáticas são obtidas através de uma consulta à tabela Ter da base de dados.

```

@login_required
def socioeconomic_detail(request, id_usuario):
    with connection.cursor() as cursor:
        # 1 Dados principais
        cursor.execute("""
            SELECT
                u.ID_USUARIO,
                u.NOME_USUARIO,
                EXTRACT(YEAR FROM AGE(CURRENT_DATE, u.DATA_NASCIMENTO)) AS idade,
                n.DESCRICAO_NACIONALIDADE,
                z.DESCRICAO_ZONA_USUARIO,
                sh.DESCRICAO_SITUACAO_HABITACIONAL,
                sl.DESCRICAO_SITUACAO_LABORAL,
                tr.NOME_TIPO_RENDIMENTO,
                ta.DESCRICAO_TIPOLOGIA,
                usec.RISCO_SOCIAL,
                usec.OBSERVACOES_USUARIO,
                usec.CRITERIO_RISCO
            FROM USUARIOS u
            JOIN USUARIOS_SOCIO_ECONOMICAS usec
              ON u.ID_USUARIO = usec.ID_USUARIO
            JOIN NACIONALIDADE n
              ON usec.ID_NACIONALIDADE = n.ID_NACIONALIDADE
            JOIN ZONA_USUARIO z
              ON u.ID_ZONA_USUARIO = z.ID_ZONA_USUARIO
            JOIN SITUACAO_HABITACIONAL sh
              ON usec.ID_SITUACAO_HABITACIONAL = sh.ID_SITUACAO_HABITACIONAL
            JOIN SITUACAO_LABORAL sl
              ON usec.ID_SITUACAO_LABORAL = sl.ID_SITUACAO_LABORAL
            JOIN TIPO_RENDIMENTO tr
              ON usec.ID_TIPO_RENDIMENTO = tr.ID_TIPO_RENDIMENTO
            JOIN TIPOLOGIA_AGREGADO_FAMILIAR ta
              ON usec.ID_TIPOLOGIA = ta.ID_TIPOLOGIA
            WHERE u.ID_USUARIO = %s
        """, [id_usuario])
        usuario = cursor.fetchone()

    if not usuario:
        return render(request, '404.html', status=404)

```

Figura 25 - Vista de Detalhar um Usuário (Dados Principais)

```

# 2 Intervenções
cursor.execute("""
    SELECT
        i.ID_INTERVENCAO,
        ti.NOME_TIPO_INTERVENCAO,
        e.NOME_ENTIDADE,
        p.PROGRAMA,
        i.DATA_INTERVENCAO,
        i.OBSERVACOES_INTERVENCAO
    FROM INTERVENCAO i
    JOIN TIPO_INTERVENCAO ti ON i.ID_TIPO_INTERVENCAO = ti.ID_TIPO_INTERVENCAO
    JOIN ENTIDADE e ON i.ID_ENTIDADE = e.ID_ENTIDADE
    JOIN PROGRAMA p ON i.ID_PROGRAMA = p.ID_PROGRAMA
    JOIN PARTICIPAR pa ON i.ID_INTERVENCAO = pa.ID_INTERVENCAO
    WHERE pa.ID_USUARIO = %s
    ORDER BY i.DATA_INTERVENCAO DESC
""", [id_usuario])
intervencoes = cursor.fetchall()
# Problemas (com período, estado e intervenção associada, se existir)
cursor.execute("""
    SELECT
        p.ID_PROBLEMA,
        p.NOME_PROBLEMA,
        t.DATA_INICIO,
        t.DATA_FIM,
        CASE WHEN t.DATA_FIM IS NULL THEN 'Ativa' ELSE 'Resolvida' END AS ESTADO,
        t.ID_INTERVENCAO,
        i.DATA_INTERVENCAO AS INTERV_DATA,
        ti.NOME_TIPO_INTERVENCAO AS INTERV_TIPO
    FROM TER t
    JOIN PROBLEMAS_IDENTIFICADAS p
      ON t.ID_PROBLEMA = p.ID_PROBLEMA
    LEFT JOIN INTERVENCAO i ON t.ID_INTERVENCAO = i.ID_INTERVENCAO
    LEFT JOIN TIPO_INTERVENCAO ti ON i.ID_TIPO_INTERVENCAO = ti.ID_TIPO_INTERVENCAO
    WHERE t.ID_USUARIO = %s
    ORDER BY t.DATA_INICIO DESC
""", [id_usuario])
problemas = cursor.fetchall()

```

Figura 26 - Vista de Detalhar um Usuário (Intervenções e Problemas)

```

# 4 Membros do agregado
cursor.execute("""
SELECT
    NOME_MEMBRO,
    ANO_NASCIMENTO,
    GRAU_PARENTESCO
FROM MEMBROS_AGREGADO_FAMILIAR
WHERE ID_UTENTE = %s
""", [id_usuario])
membros_agregado = cursor.fetchall()
# 5 Despesas fixas
cursor.execute("""
SELECT
    td.DESCRICAO_TIPO_DESPESA,
    df.VALOR_DESPESA
FROM DESPESAS_FIXAS df
JOIN TIPO_DESPESA td
ON df.ID_TIPO_DESPESA = td.ID_TIPO_DESPESA
WHERE df.ID_UTENTE = %s
""", [id_usuario])
despesas = cursor.fetchall()
# 6 Escalão de rendimentos
cursor.execute("""
SELECT
    ID_ESCALAO,
    TIPO_ESCALAO,
    VALOR_EXATO
FROM ESCALAO_RENDIMENTOS
WHERE ID_UTENTE = %s
ORDER BY ID_ESCALAO DESC
""", [id_usuario])
escaloes = cursor.fetchall()
return render(request, 'socioeconomicos/detalhe_usuario.html', {
    'usuario': usuario,
    'intervencoes': intervencoes,
    'problematicas': problematicas,
    'membros_agregado': membros_agregado,
    'despesas': despesas, 'escaloes': escaloes,})

```

Figura 27 - Vista de Detalhar um Utente (Parte final)

- Criação (socioeconomicos_novo):

Utiliza uma transação atômica (transaction.atomic()) para inserir o registo em UTENTES, em UTENTES_SOCIO_ECONOMICAS e nas tabelas relacionadas (membros, despesas, escalões). As problemáticas são inseridas na tabela TER com DATA_INICIO = data atual e DATA_FIM = NULL. A criação recorre a raw SQL para a tabela TER, uma vez que esta possui uma chave primária simples (ID_TER). São aplicadas validações rigorosas: nome obrigatório, unicidade (case-insensitive), data de nascimento não futura, obrigatoriedade de pelo menos uma problemática, e, se a tipologia do agregado não for “unipessoal”, exigência de pelo menos um membro adicional. Após a criação, é invocada a função guardar_historico_usuario que guarda um snapshot completo da entidade na tabela de histórico. As imagens seguintes apresentam a vista em python de criar um novo utente:


```

@login_required
def socioeconomicos_novo(request):
    # GET: prepara os dados para os dropdowns
    if request.method == 'GET':
        context = {
            'generos': models.Genero.objects.all(),
            'nacionalidades': models.Nacionalidade.objects.all(),
            'zonas': models.ZonaUtente.objects.all(),
            'tipologias': models.TipologiaAgregadoFamiliar.objects.all(),
            'situacoes_habitacionais': models.SituacaoHabitacional.objects.all(),
            'situacoes_laborais': models.SituacaoLaboral.objects.all(),
            'tipos_rendimento': models.TipoRendimento.objects.all(),
            'tipos_despesa': models.TipoDespesa.objects.all(),
            'todas_problematicas': models.ProblematicasIdentificadas.objects.all(),
        }
        return render(request, 'socioeconomicos/novo_utente.html', context)

    # POST: processa a criação
    if request.method == 'POST':
        # 1. Recolher dados do formulário
        nome = request.POST.get('nome_utente', '').strip()
        data_nascimento = request.POST.get('data_nascimento', '')
        id_genero = request.POST.get('genero')
        id_nacionalidade = request.POST.get('nacionalidade')
        id_zona = request.POST.get('zona')
        id_tipologia = request.POST.get('tipologia')
        id_situacao_habitacional = request.POST.get('situacao_habitacional')
        id_situacao_laboral = request.POST.get('situacao_laboral')
        id_tipo_rendimento = request.POST.get('tipo_rendimento')
        risco_social_str = request.POST.get('risco_social')
        criterio_risco = request.POST.get('criterio_risco', '').strip()
        observacoes = request.POST.get('observacoes_utente', '').strip()
        problematicas_ids = request.POST.getlist('problematicas')

```

Figura 28 - Primeira parte da view de criação

```

# 2. Validações de campos obrigatórios
errors = []

# Verificar se já existe um utente com o mesmo nome (case-insensitive)
if models.Usuarios.objects.filter(nome_utente__iexact=nome).exists():
    errors.append(f'Já existe um utente com o nome "{nome}". Por favor, utilize um nome diferente.')

if not nome:
    errors.append('Nome é obrigatório.')
if not data_nascimento:
    errors.append('Data de nascimento é obrigatória.')
else:
    # verificar se data é válida e não futura
    try:
        data_nasc = date.fromisoformat(data_nascimento)
        if data_nasc > date.today():
            errors.append('Data de nascimento não pode ser futura.')
    except ValueError:
        errors.append('Data de nascimento inválida.')
if not id_genero:
    errors.append('Sexo é obrigatório.')
if not id_nacionalidade:
    errors.append('Nacionalidade é obrigatória.')
if not id_zona:
    errors.append('Zona/Freguesia é obrigatória.')
if not id_tipologia:
    errors.append('Tipologia do agregado é obrigatória.')
if not id_situacao_habitacional:
    errors.append('Situação habitacional é obrigatória.')
if not id_situacao_laboral:
    errors.append('Situação laboral é obrigatória.')
if not id_tipo_rendimento:
    errors.append('Tipo de rendimento é obrigatório.')
if not risco_social_str:
    errors.append('Risco social é obrigatório.')
if not problematicas_ids:
    errors.append('É necessário seleccionar pelo menos uma problemática.')

```

Figura 29 - Segunda parte da view de criação

```

# 3. Validação de membros do agregado conforme tipologia
# (considerando que o utente principal não está na tabela de membros)
nomes_membros = request.POST.getlist('nome_membro')
anos_nasc = request.POST.getlist('ano_nascimento')
parentescos = request.POST.getlist('grau_parentesco')
membros_validos = []
for i in range(len(nomes_membros)):
    if nomes_membros[i].strip():
        membros_validos.append({
            'nome': nomes_membros[i].strip(),
            'ano': anos_nasc[i].strip(),
            'parentesco': parentescos[i].strip(),
        })

if id_tipologia:
    # Buscar descrição da tipologia para verificar se é unipessoal
    try:
        tipologia_obj = models.TipologiaAgregadoFamiliar.objects.get(id_tipologia=id_tipologia)
        tipologia_desc = tipologia_obj.descricao_tipologia.lower()
    except models.TipologiaAgregadoFamiliar.DoesNotExist:
        errors.append('Tipologia selecionada inválida.')
        tipologia_desc = ''

    if tipologia_desc != 'unipessoal' and not membros_validos:
        errors.append('Para esta tipologia, é necessário adicionar pelo menos um membro do agregado familiar.')
    elif tipologia_desc == 'unipessoal' and membros_validos:
        errors.append('Para esta tipologia não deve haver outros membros do agregado familiar')

# Se houver erros, reexibe o formulário com mensagens
if errors:
    for err in errors:
        messages.error(request, err)

    return redirect('socioeconomicos_novo')

```

Figura 30 - Terceira parte da view de criação

```

# 4. Se tudo válido, criar registros
risco_social = (risco_social_str == 'Em risco')

try:
    with transaction.atomic():
        # 4.1 Criar Utentes (tipo_utente = 1)
        utente = models.Utentes.objects.create(
            id_tipo_utente_id=1, # ID fixo para socioeconómico
            id_genero_id=id_genero,
            id_zona_utente_id=id_zona,
            nome_utente=nome,
            data_nascimento=data_nascimento,
            data_criacao=date.today(),
        )

        # 4.2 Criar UtentesSocioEconomicas
        socio = models.UtentesSocioEconomicas.objects.create(
            id_utente=utente,
            id_nacionalidade_id=id_nacionalidade,
            id_tipologia_id=id_tipologia,
            id_situacao_habitacional_id=id_situacao_habitacional,
            id_situacao_laboral_id=id_situacao_laboral,
            id_tipo_rendimento_id=id_tipo_rendimento,
            risco_social=risco_social,
            observacoes_utente=observacoes,
            criterio_risco=criterio_risco,
        )

        # 4.3 Membros do agregado
        for membro in membros_validos:
            models.MembrosAgregadoFamiliar.objects.create(
                id_utente=socio,
                nome_membro=membro['nome'],
                ano_nascimento=int(membro['ano']) if membro['ano'] else None,
                grau_parentesco=membro['parentesco'],
            )

```

Figura 31 - Quarta parte da view de criação

```

# 4.4 Despesas fixas (se houver)
tipos_despesa = request.POST.getlist('tipo_despesa')
valores_despesa = request.POST.getlist('valor_despesa')
for i in range(len(tipos_despesa)):
    tipo_id = tipos_despesa[i]
    valor = valores_despesa[i]
    if tipo_id and valor:
        models.DespesasFixas.objects.create(
            id_tipo_despesa_id=tipo_id,
            id_utente=socio,
            valor_despesa=float(valor),
        )

# 4.5 Escalões de rendimento (se houver)
tipos_escalao = request.POST.getlist('tipo_escalao')
valores_escalao = request.POST.getlist('valor_exato')
for i in range(len(tipos_escalao)):
    tipo = tipos_escalao[i]
    valor = valores_escalao[i]
    if tipo and valor:
        models.EscalaoRendimentos.objects.create(
            id_utente=socio,
            tipo_escalao=tipo,
            valor_exato=float(valor),
        )

# 4.6 Problemáticas
data_atual = date.today()
nova_prob_texto = request.POST.get('nova_problematika', '').strip()
if nova_prob_texto:
    nova_prob = models.ProblematicasIdentificadas.objects.create(nome_problematika=nova_prob_texto)
    novo_id = nova_prob.id_problematika
    problematicas_ids.append(str(novo_id))
with connection.cursor() as cursor:
    for pid in problematicas_ids:
        cursor.execute(
            "INSERT INTO ter (id_utente, id_problematika, data_inicio, data_fim) VALUES (%s, %s, %s, %s)",
            [socio.id_utente_id, pid, data_atual, None]
        )

```

Figura 32 - Quinta parte da view de criação

```

    guardar_historico_utente(request, utente.id_utente)

    messages.success(request, f'Utente {nome} criado com sucesso!')
    return redirect('socioeconomicos_lista')

except Exception as e:
    messages.error(request, f'Erro ao criar utente: {str(e)}')
    # rollback automático

# Fallback
return redirect('socioeconomicos_lista')

```

Figura 33 - Parte final da view de criação

- Edição (socioeconomicos_editar):

Utiliza uma transação atômica (transaction.atomic()) para inserir o registo em UTENTES, em UTENTES_SOCIO_ECONOMICAS e nas tabelas relacionadas (membros, despesas, escalões). As problemáticas são inseridas na tabela TER com DATA_INICIO = data atual e DATA_FIM = NULL. A criação recorre a raw SQL para a tabela TER, uma vez que esta possui uma chave primária simples (ID_TER). São aplicadas validações rigorosas: nome obrigatório, unicidade (case-insensitive), data de nascimento não futura, obrigatoriedade de pelo menos uma problemática, e, se a tipologia

do agregado não for “unipessoal”, exigência de pelo menos um membro adicional. Após a criação, é invocada a função `guardar_historico_utente` que guarda um snapshot completo da entidade na tabela de histórico. As imagens seguintes mostram essa view de edição:

```
@login_required
def socioeconomicos_editar(request, id_utente):
    # Obter o utente base e o registo socioeconómico
    utente = get_object_or_404(models.Utentes, pk=id_utente)
    socio = get_object_or_404(models.UtentesSocioEconomicas, pk=id_utente)

    if request.method == 'GET':
        # Preparar os dados existentes para preencher o formulário
        membros = models.MembrosAgregadoFamiliar.objects.filter(id_utente=socio).values('nome_membro', 'ano_nascimento', 'grau_parentesco')
        despesas = models.DespesasFixas.objects.filter(id_utente=socio).select_related('id_tipo_despesa').values('id_tipo_despesa_id', 'valor_despesa')
        escaloes = models.EscalaoRendimentos.objects.filter(id_utente=socio).values('tipo_escalao', 'valor_exato')
        problematicas_ids = models.Ter.objects.filter(id_utente=socio, data_fim_isnull=True).values_list('id_problematica_id', flat=True)
        # Intervenções do utente para associar à resolução
        intervencoes_utente = models.Intervencao.objects.filter(participar_id_utente=socio).order_by('-data_intervencao').distinct()

        # Montar estrutura semelhante ao 'submetido' do POST
        submetido = {
            'nome_utente': utente.nome_utente,
            'data_nascimento': utente.data_nascimento.isoformat() if utente.data_nascimento else '',
            'genero': utente.id_genero_id,
            'nacionalidade': socio.id_nacionalidade_id,
            'zona': utente.id_zona_utente_id,
            'tipologia': socio.id_tipologia_id,
            'situacao_habitacional': socio.id_situacao_habitacional_id,
            'situacao_laboral': socio.id_situacao_laboral_id,
            'tipo_rendimento': socio.id_tipo_rendimento_id,
            'risco_social': 'Em risco' if socio.risco_social else 'Situação estável',
            'criterio_risco': socio.criterio_risco or '',
            'observacoes_utente': socio.observacoes_utente or '',
            'problematicas': [str(pid) for pid in problematicas_ids],
        }

        membros_validos = [
            {'nome': m['nome_membro'], 'ano': m['ano_nascimento'], 'parentesco': m['grau_parentesco']}
            for m in membros
        ]
        despesas_validas = [
            {'tipo': d['id_tipo_despesa_id'], 'valor': d['valor_despesa']}
            for d in despesas
        ]

        escaloes_validos = [
            {'tipo': e['tipo_escalao'], 'valor': e['valor_exato']}
            for e in escaloes
        ]

        context = {
            'generos': models.Genero.objects.all(),
            'nacionalidades': models.Nacionalidade.objects.all(),
            'zonas': models.ZonaUtente.objects.all(),
            'tipologias': models.TipologiaAgregadoFamiliar.objects.all(),
            'situacoes_habitacionais': models.SituacaoHabitacional.objects.all(),
            'situacoes_laborais': models.SituacaoLaboral.objects.all(),
            'tipos_rendimento': models.TipoRendimento.objects.all(),
            'tipos_despesa': models.TipoDespesa.objects.all(),
            'todas_problematicas': models.ProblematicasIdentificadas.objects.all(),
            'submetido': submetido,
            'membros_submetidos': membros_validos,
            'despesas_submetidas': despesas_validas,
            'escaloes_submetidos': escaloes_validos,
            'intervencoes_utente': intervencoes_utente,
            'id_utente': id_utente,
        }

    return render(request, 'socioeconomicos/editar_utente.html', context)

# POST - atualizar dados
if request.method == 'POST':
    # Recolher dados do formulário (igual à criação)
    nome = request.POST.get('nome_utente', '').strip()
    data_nascimento = request.POST.get('data_nascimento', '')
    id_genero = request.POST.get('genero')
    id_nacionalidade = request.POST.get('nacionalidade')
    id_zona = request.POST.get('zona')
    id_tipologia = request.POST.get('tipologia')
    id_situacao_habitacional = request.POST.get('situacao_habitacional')
    id_situacao_laboral = request.POST.get('situacao_laboral')
    id_tipo_rendimento = request.POST.get('tipo_rendimento')
    risco_social_str = request.POST.get('risco_social')
    criterio_risco = request.POST.get('criterio_risco', '').strip()
```

Figura 34 - Primeira parte da view de edição

```
escaloes_validos = [
    {'tipo': e['tipo_escalao'], 'valor': e['valor_exato']}
    for e in escaloes
]

context = {
    'generos': models.Genero.objects.all(),
    'nacionalidades': models.Nacionalidade.objects.all(),
    'zonas': models.ZonaUtente.objects.all(),
    'tipologias': models.TipologiaAgregadoFamiliar.objects.all(),
    'situacoes_habitacionais': models.SituacaoHabitacional.objects.all(),
    'situacoes_laborais': models.SituacaoLaboral.objects.all(),
    'tipos_rendimento': models.TipoRendimento.objects.all(),
    'tipos_despesa': models.TipoDespesa.objects.all(),
    'todas_problematicas': models.ProblematicasIdentificadas.objects.all(),
    'submetido': submetido,
    'membros_submetidos': membros_validos,
    'despesas_submetidas': despesas_validas,
    'escaloes_submetidos': escaloes_validos,
    'intervencoes_utente': intervencoes_utente,
    'id_utente': id_utente,
}

return render(request, 'socioeconomicos/editar_utente.html', context)

# POST - atualizar dados
if request.method == 'POST':
    # Recolher dados do formulário (igual à criação)
    nome = request.POST.get('nome_utente', '').strip()
    data_nascimento = request.POST.get('data_nascimento', '')
    id_genero = request.POST.get('genero')
    id_nacionalidade = request.POST.get('nacionalidade')
    id_zona = request.POST.get('zona')
    id_tipologia = request.POST.get('tipologia')
    id_situacao_habitacional = request.POST.get('situacao_habitacional')
    id_situacao_laboral = request.POST.get('situacao_laboral')
    id_tipo_rendimento = request.POST.get('tipo_rendimento')
    risco_social_str = request.POST.get('risco_social')
    criterio_risco = request.POST.get('criterio_risco', '').strip()
```

Figura 35 - Segunda parte da view de edição

```

observacoes = request.POST.get('observacoes_usuario', '').strip()
problematicas_ids = request.POST.getlist('problematicas')

# Validações (reaproveitar lógica da criação)
errors = []
if not nome:
    errors.append('Nome é obrigatório.')
if not data_nascimento:
    errors.append('Data de nascimento é obrigatória.')
else:
    try:
        data_nasc = date.fromisoformat(data_nascimento)
        if data_nasc > date.today():
            errors.append('Data de nascimento não pode ser futura.')
    except ValueError:
        errors.append('Data de nascimento inválida.')
if not id_genero:
    errors.append('Sexo é obrigatório.')
if not id_nacionalidade:
    errors.append('Nacionalidade é obrigatória.')
if not id_zona:
    errors.append('Zona/Freguesia é obrigatória.')
if not id_tipologia:
    errors.append('Tipologia do agregado é obrigatória.')
if not id_situacao_habitacional:
    errors.append('Situação habitacional é obrigatória.')
if not id_situacao_laboral:
    errors.append('Situação laboral é obrigatória.')
if not id_tipo_rendimento:
    errors.append('Tipo de rendimento é obrigatório.')
if not risco_social_str:
    errors.append('Risco social é obrigatório.')

resolucoes = {}
for key, value in request.POST.items():
    if key.startswith('resolucao_intervencao_') and value:
        pid = key.split('.')[1]
        resolucoes[pid] = value

```

Figura 36 - Terceira parte da view de edição

```

# Membros
nomes_membros = request.POST.getlist('nome_membro')
anos_nasc = request.POST.getlist('ano_nascimento')
parentescos = request.POST.getlist('grau_parentesco')
membros_validos = []
for i in range(len(nomes_membros)):
    if nomes_membros[i].strip():
        membros_validos.append({
            'nome': nomes_membros[i].strip(),
            'ano': anos_nasc[i].strip(),
            'parentesco': parentescos[i].strip(),
        })

if id_tipologia:
    try:
        tipologia_obj = models.TipologiaAgregadoFamiliar.objects.get(id_tipologia=id_tipologia)
        tipologia_desc = tipologia_obj.descricao_tipologia.lower()
    except models.TipologiaAgregadoFamiliar.DoesNotExist:
        errors.append('Tipologia selecionada inválida.')
        tipologia_desc = ''
    if tipologia_desc != 'unipessoal' and not membros_validos:
        errors.append('Para esta tipologia, é necessário adicionar pelo menos um membro do agregado familiar.')
    elif tipologia_desc == 'unipessoal' and membros_validos:
        errors.append('Para esta tipologia não deve haver outros membros do agregado familiar')

if errors:
    for err in errors:
        messages.error(request, err)
    # Redirecionar para a página de edição (GET)
    return redirect('socioeconomicos_editar', id_usuario=id_usuario)

guardar_historico_usuario(request, usuario.id_usuario)

# Se passou nas validações, atualizar
risco_social = (risco_social_str == 'Em risco')

```

Figura 37 - Quarta parte da view de edição

```

try:
    with transaction.atomic():
        # 1. Atualizar dados básicos do utente
        utente.nome_utente = nome
        utente.data_nascimento = data_nascimento
        utente.id_genero_id = id_genero
        utente.id_zona_utente_id = id_zona
        utente.save()

        # 2. Atualizar dados socioeconômicos
        socio.id_nacionalidade_id = id_nacionalidade
        socio.id_tipologia_id = id_tipologia
        socio.id_situacao_habitacional_id = id_situacao_habitacional
        socio.id_situacao_laboral_id = id_situacao_laboral
        socio.id_tipo_rendimento_id = id_tipo_rendimento
        socio.risco_social = risco_social
        socio.criterio_risco = criterio_risco
        socio.observacoes_utente = observacoes
        socio.save()

        # 3. Membros - apagar todos e recriar (simples)
        models.MembrosAgregadoFamiliar.objects.filter(id_utente=socio).delete()
        for m in membros_validos:
            models.MembrosAgregadoFamiliar.objects.create(
                id_utente=socio,
                nome_membro=m['nome'],
                ano_nascimento=int(m['ano']) if m['ano'] else None,
                grau_parentesco=m['parentesco'],
            )

        # 4. Despesas fixas - apagar e recriar
        models.DespesasFixas.objects.filter(id_utente=socio).delete()
        tipos_despesa = request.POST.getlist('tipo_despesa')
        valores_despesa = request.POST.getlist('valor_despesa')
        for i in range(len(tipos_despesa)):
            tipo_id = tipos_despesa[i]
            valor = valores_despesa[i]
            if tipo_id and valor:

```

Figura 38 - Quinta parte da view de edição

```

        models.DespesasFixas.objects.create(
            id_tipo_despesa_id=tipo_id,
            id_utente=socio,
            valor_despesa=float(valor),
        )

    # 5. Escalões - apagar e recriar
    models.EscalaoRendimentos.objects.filter(id_utente=socio).delete()
    tipos_escalao = request.POST.getlist('tipo_escalao')
    valores_escalao = request.POST.getlist('valor_exato')
    for i in range(len(tipos_escalao)):
        tipo = tipos_escalao[i]
        valor = valores_escalao[i]
        if tipo and valor:
            models.EscalaoRendimentos.objects.create(
                id_utente=socio,
                tipo_escalao=tipo,
                valor_exato=float(valor),
            )

    # 6. Problemáticas - atualizar com datas

    # Campo aberto: nova problemática (opcional, sem separadores)
    nova_prob_texto = request.POST.get('nova_problematICA', '').strip()
    if nova_prob_texto:
        nova_prob = models.ProblematicasIdentificadas.objects.create(nome_problematICA=nova_prob_texto)
        novo_id = nova_prob.id_problematICA
        problemáticas_ids.append(str(novo_id))

    # 4. Problemáticas - atualizar com datas e associação a intervenção
    # Obter IDs das problemáticas atualmente ativas
    with connection.cursor() as cursor:
        cursor.execute(
            "SELECT id_problematICA FROM ter WHERE id_utente = %s AND data_fim IS NULL",
            [socio.id_utente_id]
        )
        atuais = {row[0] for row in cursor.fetchall()}

```

Figura 39 - Sexta parte da view de edição

```

novas = set(int(pid) for pid in problemáticas_ids)
removidas = atuais - novas
adicionadas = novas - atuais

# Resolver as removidas: data_fim = hoje, e opcionalmente id_intervencao
if removidas:
    with connection.cursor() as cursor:
        for pid in removidas:
            id_interv = resolucoes.get(str(pid))
            cursor.execute(
                "UPDATE ter SET data_fim = %, id_intervencao = % WHERE id_usuario = % AND id_problema = % AND data_fim IS NULL",
                [date.today(), id_interv if id_interv else None, socio.id_usuario_id, pid]
            )

# Adicionar novas problemáticas (data_inicio = hoje)
if adicionadas:
    with connection.cursor() as cursor:
        for pid in adicionadas:
            cursor.execute(
                "INSERT INTO ter (id_usuario, id_problema, data_inicio, data_fim, id_intervencao) VALUES (%s, %s, %s, %s, %s)",
                [socio.id_usuario_id, pid, date.today(), None, None]
            )

messages.success(request, f'Utente (nome) atualizado com sucesso!')
return redirect('socioeconomicos_detalhe', id_usuario=id_usuario)

except Exception as e:
    messages.error(request, f'Erro ao atualizar utente: {str(e)}')

return redirect('socioeconomicos_lista')

```

Figura 40 - Parte final da view de edição

- Eliminação (socioeconomicos_eliminar):

Verifica a existência de intervenções associadas (através da tabela PARTICIPAR). Se existirem, a eliminação é bloqueada e é exibida uma mensagem de erro. Caso contrário, a transação atômica apaga os registos por ordem inversa de dependência (terminando com a tabela UTENTES). A view a seguir serve para eliminar as informações de um certo utente:

```

@login_required
def socioeconomicos_eliminar(request, id_usuario):
    # Verificar se o utente existe (usando raw SQL)
    with connection.cursor() as cursor:
        cursor.execute("SELECT ID_UTENTE, NOME_UTENTE FROM UTENTES WHERE ID_UTENTE = %s", [id_usuario])
        utente = cursor.fetchone()
        if not utente:
            messages.error(request, 'Utente não encontrado.')
            return redirect('socioeconomicos_lista')

    # Verificar se existem intervenções associadas (tabela PARTICIPAR)
    cursor.execute("SELECT COUNT(*) FROM PARTICIPAR WHERE ID_UTENTE = %s", [id_usuario])
    count_intervencoes = cursor.fetchone()[0]
    if count_intervencoes > 0:
        messages.error(request, f'Não é possível eliminar o utente "{utente[1]}" porque tem {count_intervencoes} intervenção(ões) associada(s).')
        return redirect('socioeconomicos_detalhe', id_usuario=id_usuario)

    # Se não houver intervenções, prosseguir com a eliminação
    try:
        with transaction.atomic():
            with connection.cursor() as cursor:
                # Apagar registos dependentes (ordem inversa das FK)
                cursor.execute("DELETE FROM TER WHERE ID_UTENTE = %s", [id_usuario])
                cursor.execute("DELETE FROM MEMBROS_AGREGADO_FAMILIAR WHERE ID_UTENTE = %s", [id_usuario])
                cursor.execute("DELETE FROM DESPESAS_FIXAS WHERE ID_UTENTE = %s", [id_usuario])
                cursor.execute("DELETE FROM ESCALAO_BENEFICIOS WHERE ID_UTENTE = %s", [id_usuario])
                cursor.execute("DELETE FROM UTENTES_SOCIO_ECONOMICAS WHERE ID_UTENTE = %s", [id_usuario])
                cursor.execute("DELETE FROM UTENTES WHERE ID_UTENTE = %s", [id_usuario])

            messages.success(request, f'Utente "{utente[1]}" eliminado com sucesso.')
    except Exception as e:
        messages.error(request, f'Erro ao eliminar utente: {str(e)}')
        return redirect('socioeconomicos_detalhe', id_usuario=id_usuario)

    return redirect('socioeconomicos_lista')

```

Figura 41 - View de eliminar

- CRUD de Idosos: O módulo de gestão de utentes idosos foi concebido para dar resposta às necessidades específicas do acompanhamento de pessoas com idade

igual ou superior a 65 anos, abrangendo áreas como a identificação pessoal, a situação de autonomia e funcionalidade, a saúde, o bem-estar emocional e social, as condições habitacionais e o registo de contactos efetuados pela equipa técnica. A implementação seguiu os mesmos princípios de atomicidade e validação utilizados no módulo de utentes socioeconómicos, mas com uma estrutura de dados mais complexa devido ao maior número de tabelas associadas e à necessidade de manter um histórico rigoroso de evolução (ex: quedas, agravamento da dependência, isolamento). De seguida, descrevem-se as cinco operações fundamentais que compõem o CRUD. A seguir, descrevem-se as operações principais:

- Listagem (idosos_lista): A página de listagem de idosos apresenta uma tabela paginada (10 registos por página) com as colunas: código, nome, idade (calculada a partir da data de nascimento), sexo, estado civil, “vive sozinho”, zona e ações (ver, editar, eliminar). Foi implementado um formulário de filtro por nome (case-insensitive), que adiciona uma cláusula `WHERE u.NOME_UTENTE ILIKE %s` à consulta SQL base. A paginação é gerida pelo Paginador do Django, mantendo o termo de pesquisa nos links de navegação. A ordenação é feita por `ID_UTENTE` para garantir uma sequência estável. Os dados são obtidos através de uma consulta que faz `JOIN` entre as tabelas `UTENTES`, `UTENTES_IDOSOS`, `GENERO`, `ESTADO_CIVIL` e `ZONA_UTENTE`. A utilização de `ILIKE` (em vez de `LIKE`) torna a pesquisa insensível a maiúsculas/minúsculas, melhorando a experiência do utilizador.

- Detalhe (idosos_detalhe): A página de detalhe de um idoso reúne toda a informação disponível sobre o utente, organizada em vários cartões temáticos para facilitar a consulta. Um primeiro cartão exibe a informação pessoal: nome, idade, sexo, zona/freguesia, estado civil, “vive sozinho”, contacto de emergência e familiar/cuidador de referência. Seguem-se os cartões de saúde e autonomia (grau de autonomia, mobilidade, medicação diária, dificuldade de acesso a cuidados de saúde, acompanhamento regular no centro de saúde) e de bem-estar e social (nível de isolamento, relação com a família, frequência de contacto social, participação em atividades da freguesia, sinais de tristeza/depressão, luto recente). O cartão de habitação e segurança inclui o tipo de habitação, aquecimento adequado no inverno, segurança (risco de quedas, iluminação) e necessidade de adaptações. Para

os dados multidimensionais, são apresentadas secções específicas: atividades da vida diária (AVD) como uma lista de textos descritivos; doenças crónicas, barreiras arquitetónicas e sinais de fragilidade como etiquetas (badges) coloridas; histórico de contactos numa tabela com data, tipo, frequência e observações, ordenada da mais recente para a mais antiga; e registo de quedas numa tabela com data e observações, também ordenada decrescentemente. Todas as consultas são escritas em SQL puro, recorrendo a LEFT JOIN com as tabelas de dicionário para obter as descrições textuais dos códigos armazenados. A consulta principal envolve as tabelas UTENTES, UTENTES_IDOSOS, GENERO, ZONA_UTENTE, ESTADO_CIVIL, GRAU_AUTONOMIA, MOBILIDADE, NIVEL_ISOLAMENTO, RELACAO_FAMILIA, FREQUENCIA_CONTACTO_SOCIAL e TIPO_HABITACAO. A idade é calculada no momento da consulta usando EXTRACT(YEAR FROM AGE(CURRENT_DATE, u.DATA_NASCIMENTO)). Esta abordagem garante que a idade apresentada é sempre a atual, sem necessidade de armazenamento redundante.

- Criação (idosos_novo): O formulário de criação de um novo idoso é extenso e está dividido em várias secções, cada uma correspondendo a um grupo de atributos. A recolha dos dados é feita por POST e toda a operação de inserção é envolvida numa transação atómica (transaction.atomic()) para garantir a consistência da base de dados. As validações implementadas incluem a verificação de todos os campos obrigatórios (nome, data de nascimento, género, zona, estado civil, grau de autonomia, mobilidade, nível de isolamento, relação com a família, frequência de contacto social, tipo de habitação), a validação da data de nascimento (formato e não futura) e a proibição de nomes duplicados (case-insensitive). Para as listas dinâmicas, as doenças crónicas, barreiras arquitetónicas e sinais de fragilidade são apresentadas como checkboxes com opções pré-definidas. Foram ainda disponibilizados campos de texto livre (“nova doença”, “nova barreira”, “nova fragilidade”) que permitem ao técnico adicionar um novo valor à respetiva tabela dicionário em tempo real. Se o campo for preenchido, a aplicação verifica se o valor já existe (case-insensitive); se não existir, cria um novo registo e adiciona o seu ID à lista de selecionados. As quedas são

geridas como linhas dinâmicas: o técnico pode adicionar quantas ocorrências desejar, cada uma com data e observações, sendo as linhas vazias ignoradas. Após a validação, a transação atômica executa os seguintes passos: (1) criação de um registo na tabela UTENTES (com `id_tipo_utente = 2` e `data_criacao = data atual`); (2) criação do registo correspondente em UTENTES_IDOSOS; (3) inserção de um único registo na tabela ATIVIDADES_DIARIA para as atividades da vida diária; (4) inserção em DOENCAS_IDOSOS, BARREIRAS_IDOSOS e FRAGILIDADE_IDOSO com raw SQL (devido à chave primária composta), percorrendo as listas de IDs selecionados; (5) inserção individual de cada queda na tabela QUEDAS_IDOSO. Por fim, é guardado um snapshot completo do idoso na tabela `historico_idoso` através da função `guardar_historico_idoso`, que serializa todos os dados em JSONB.

- Edição (`idosos_editar`): A edição de um idoso carrega previamente todos os dados atuais para o formulário, incluindo as listas de doenças, barreiras, fragilidades e quedas já registadas. A lógica de atualização também é envolvida numa transação atômica. Para as tabelas de muitos-para-muitos (`DOENCAS_IDOSOS`, `BARREIRAS_IDOSOS`, `FRAGILIDADE_IDOSO`) e para as quedas, optou-se pela abordagem apagar todos os registos existentes e recriar com os novos dados submetidos. Esta estratégia simplifica o código e evita a necessidade de comparar conjuntos de identificadores para determinar quais adicionar, remover ou manter. A tabela `ATIVIDADES_DIARIA` (que guarda um único texto descritivo) é atualizada ou criada/apagada consoante o campo esteja preenchido ou não. Os campos de texto livre para novas doenças, barreiras ou fragilidades são processados da mesma forma que na criação: se preenchidos, o valor é adicionado à lista de IDs selecionados antes da recriação. Antes de qualquer alteração, é chamada a função `guardar_historico_idoso`, que armazena o estado atual do utente na tabela de histórico, garantindo a rastreabilidade das modificações. No que respeita às quedas, a estratégia “apagar e recriar” é particularmente útil porque elimina a necessidade de controlar quais linhas foram removidas ou adicionadas individualmente. Após a atualização, o utilizador é redirecionado para a página de detalhe do idoso.

- Eliminação (idosos_eliminar): A eliminação de um idoso é permitida apenas se não existirem contactos associados (tabela CONTACTO). A verificação é feita através de uma consulta SQL que conta o número de contactos do utente. Caso existam, a operação é bloqueada e é apresentada uma mensagem de erro, informando o técnico de que deve primeiro apagar os contactos ou optar por manter o idoso inativo. Esta restrição evita a perda accidental de um histórico relevante de acompanhamento. Quando a eliminação é possível, a transação atómica apaga, pela ordem inversa das dependências, os registos nas tabelas CONTACTO, HISTORICO_IDOSO, QUEDAS_IDOSO, ATIVIDADES_DIARIA, FRAGILIDADE_IDOSO, BARREIRAS_IDOSOS, DOENCAS_IDOSOS, UTENTES_IDOSOS e finalmente UTENTES. A eliminação em cascata é feita manualmente para garantir o controlo sobre a integridade referencial, uma vez que as restrições ON DELETE RESTRICT impediriam a eliminação se a ordem não fosse respeitada. Após a conclusão, é apresentada uma mensagem de sucesso e o utilizador é redirecionado para a lista de idosos.

- Intervenções: O módulo de intervenções constitui um dos pilares da plataforma, uma vez que regista todas as ações realizadas no âmbito do acompanhamento dos utentes socioeconómicos. Cada intervenção pode envolver vários utentes (através da tabela de associação PARTICIPAR) e pode estar na origem de um ou mais encaminhamentos para entidades da rede social. O desenho deste módulo teve em conta a necessidade de flexibilidade (permitir adicionar novos tipos, entidades ou programas em tempo real) e de eficiência (lidar com um elevado número de intervenções ao longo do tempo). De seguida, descrevem-se as cinco operações fundamentais que compõem o CRUD de intervenções.
 - Listagem de Intervenções (intervencoes_lista): A página de listagem de intervenções apresenta uma tabela paginada (10 registos por página) com as seguintes colunas: ID, data/hora, tipo de intervenção, entidade, programa, administrador responsável, número de utentes participantes e ações (ver, editar, eliminar). A ordenação é decrescente por data, garantindo que as intervenções mais recentes surgem no topo. Não foi implementado um filtro por texto, uma vez que o número total de intervenções é, em geral, moderado; no entanto, a paginação assegura uma boa experiência de consulta mesmo quando existem centenas de registos. A consulta SQL

principal recorre a JOIN com as tabelas TIPO_INTERVENCAO, ENTIDADE, PROGRAMA e ADMINISTRADORES, e inclui uma subconsulta para contar os utentes participantes (SELECT COUNT(*) FROM PARTICIPAR WHERE ID_INTERVENCAO = i.ID_INTERVENCAO). Os dados são obtidos através de connection.cursor() e paginados com o Paginator do Django.

- Detalhe (detalhe_intervencao): A página de detalhe de uma intervenção exibe todas as informações relevantes, organizadas em dois cartões principais. O primeiro cartão contém os dados gerais da intervenção: identificador, administrador responsável, tipo, data/hora, entidade, programa e observações. O segundo cartão apresenta a lista de utentes participantes, com o código, nome, idade, sexo, zona e risco social de cada um. Cada utente possui um link para a sua própria ficha de detalhe, permitindo uma navegação rápida entre registos relacionados. Para além destes, foi adicionada uma secção que lista os encaminhamentos associados à intervenção (se existirem), com o ID, entidade, tipo de resposta, estado, data de resposta, tempo de resposta e um link para o detalhe do encaminhamento. Esta integração permite ao técnico consultar, a partir da intervenção, todo o fluxo de resposta da rede social. As consultas são realizadas em SQL puro, com LEFT JOIN para as tabelas de dicionário, e ordenadas por data decrescente no caso dos encaminhamentos.
- Criar (criar_intervencao) - O formulário de criação de uma nova intervenção está dividido em duas partes principais. Na primeira, o técnico seleciona ou introduz o tipo de intervenção, a entidade e o programa. Cada um destes campos é composto por um elemento <select> com as opções pré-definidas (extraídas das respetivas tabelas dicionário) e, imediatamente abaixo, um campo de texto livre com o rótulo “Ou adicione um novo ...”. Se o técnico preencher o campo de texto, o novo valor é criado na tabela dicionário apropriada (ex: TIPO_INTERVENCAO) e o seu ID é utilizado na intervenção. Esta abordagem permite uma grande flexibilidade, pois a entidade de acolhimento pode acrescentar novos tipos de ação à medida que as práticas evoluem, sem necessidade de intervenção do programador. A segunda parte do formulário consiste na seleção dos utentes participantes,

originalmente apresentada como uma tabela com checkboxes. Para lidar com um elevado número de utentes (centenas ou milhares), esta tabela foi substituída por um campo <select multiple> estilizado com a biblioteca Select2, que oferece pesquisa instantânea e seleção múltipla intuitiva. A lista de opções é carregada integralmente (limitada a 500 registos, o que é aceitável) e o utilizador pode filtrar por nome ou código. Antes da inserção, são aplicadas validações: a data da intervenção é obrigatória, é necessário seleccionar pelo menos um utente, e tem de existir um tipo, entidade e programa (seja através do dropdown ou do campo de texto livre). A criação é envolvida numa transação atómica, que executa os seguintes passos: (1) se o campo de texto livre para o tipo de intervenção foi preenchido, cria-se um novo registo em TIPO_INTERVENCAO e utiliza-se o seu ID; o mesmo para entidade e programa; (2) cria-se o registo na tabela INTERVENCAO com o administrador autenticado; (3) para cada ID de utente seleccionado, insere-se um registo na tabela PARTICIPAR utilizando raw SQL (devido à chave primária composta). Após o sucesso, o utilizador é redireccionado para a página de detalhe da intervenção recém-criada.

- Editar (editar_intervencao) - A edição de uma intervenção segue a mesma lógica da criação, mas com pré-preenchimento dos dados atuais. O formulário carrega os valores do tipo, entidade, programa, data e observações, bem como os utentes atualmente associados (pré-seleccionados no Select2). Tal como na criação, o técnico pode optar por adicionar novos tipos, entidades ou programas através dos campos de texto livre; estes novos valores são criados no momento da submissão e substituem os anteriores. A actualização dos utentes participantes é feita através da estratégia “apagar todos os registos em PARTICIPAR e recriar com a nova lista”. Esta abordagem simplifica a lógica e garante que a associação reflete exactamente o que o utilizador seleccionou. A operação é envolvida numa transação atómica, assegurando que, se algum passo falhar, nenhuma alteração é persistida. Após a actualização, o utilizador é redireccionado para o detalhe da intervenção.
- Eliminar (eliminar_intervencao) – A eliminação de uma intervenção é condicionada à existência de encaminhamentos associados. Antes de apagar,

a view verifica se existem registos na tabela ENCAMINHAMENTOS_REDE que referenciem esta intervenção. Se existirem, a operação é bloqueada e uma mensagem de erro informa o utilizador de que deve primeiro eliminar os encaminhamentos (ou reatribuí-los). Esta verificação evita a perda de informações sobre a resposta da rede social. Se não existirem encaminhamentos, a transação atómica executa dois DELETE consecutivos: primeiro em PARTICIPAR (removendo todas as associações com utentes) e depois em INTERVENCAO. A eliminação é simples, uma vez que a base de dados está configurada com “ON DELETE RESTRICT” nas chaves estrangeiras, pelo que a ordem manual garante o cumprimento das restrições. Após a eliminação, o utilizador é redirecionado para a lista de intervenções.

- Encaminhamentos: O módulo de encaminhamentos regista as respostas da rede social às intervenções realizadas, permitindo acompanhar o fluxo de encaminhamentos para entidades parceiras (Junta de Freguesia, IPSS, SNS, Segurança Social, Banco Alimentar, etc.) e avaliar a eficácia da resposta. Cada encaminhamento associa-se a uma intervenção existente, a uma entidade e a um tipo de resposta (aceite, recusada, em espera, lista de espera, não se aplica). O desenho deste módulo procurou conciliar a necessidade de uma seleção eficiente da intervenção (dado o elevado número potencial) com a simplicidade de preenchimento dos restantes campos.
 - Listagem (encaminhamentos_lista) - A página de listagem de encaminhamentos apresenta uma tabela paginada (10 registos por página) com as colunas: ID, data da resposta, entidade, tipo de resposta, estado do encaminhamento (sim/não), tempo de resposta (dias), intervenção associada (com link para o detalhe) e ações (ver, editar, eliminar). A ordenação é decrescente por data de resposta, com os registos sem data a surgir no final (ORDER BY e.DATA_RESPOSTA DESC NULLS LAST). A consulta SQL principal recorre a JOIN com as tabelas ENTIDADE, TIPO_RESPOSTA e INTERVENCAO, obtendo também o ID da intervenção para construir o link. A paginação é gerida pelo Paginator do Django.

- Detalhe (encaminhamentos_detalhe) - A página de detalhe de um encaminhamento exibe todas as informações relevantes, organizadas em dois cartões principais. O primeiro cartão contém os dados do encaminhamento: ID, entidade, tipo de resposta, descrição, estado (encaminhamento feito ou não), data da resposta, tempo de resposta e observações. O segundo cartão apresenta os dados da intervenção associada, incluindo ID, data, tipo, entidade, programa e administrador responsável, com um link para a página de detalhe da intervenção. Para além disso, são listados os utentes participantes nessa intervenção (código, nome, idade, sexo, zona e risco social), com ligação para as respetivas fichas de utente. Esta visão integrada permite ao técnico compreender o contexto completo do encaminhamento, desde a ação inicial até à resposta da entidade. As consultas são escritas em SQL puro, com LEFT JOIN para as tabelas de dicionário, e a listagem de utentes é ordenada alfabeticamente.
- Criação (encaminhamento_novo) - O formulário de criação de um novo encaminhamento está dividido em duas partes. Na primeira, o técnico deve selecionar a intervenção à qual o encaminhamento se refere. Devido à possibilidade de existirem milhares de intervenções, optou-se por um campo de pesquisa com autocomplete (Select2 com AJAX) em vez de uma tabela ou de um dropdown tradicional. O utilizador digita parte do ID, data, tipo, entidade, programa ou responsável, e o sistema devolve as intervenções mais relevantes (limitadas a 20). Esta abordagem garante uma experiência rápida e eficiente, mesmo com uma base de dados muito extensa. A segunda parte do formulário contém os restantes campos: entidade (obrigatória, selecionada a partir de um dropdown com as entidades pré-definidas), tipo de resposta (opcional, selecionado a partir de um dropdown), descrição do encaminhamento (obrigatória, campo de texto), encaminhamento realizado? (checkbox), data da resposta (opcional, campo datetime-local) e tempo de resposta (opcional, número inteiro de dias). São aplicadas validações simples: intervenção obrigatória, entidade obrigatória, descrição obrigatória. A criação é envolvida numa transação atômica, utilizando o ORM para criar o registo em ENCAMINHAMENTOS_REDE (uma vez que esta tabela tem uma chave primária simples). Após a criação, o utilizador é redirecionado para a página de detalhe do novo encaminhamento.

- Encaminhamentos (encaminhamentos_editar) - A edição de um encaminhamento carrega previamente todos os dados atuais para o formulário, incluindo a intervenção associada (que é apresentada no campo Select2 com o texto descritivo da intervenção). O técnico pode alterar qualquer um dos campos, incluindo a intervenção (pesquisando novamente através do autocomplete). As validações são idênticas às da criação. A atualização é feita através do ORM, envolvida numa transação atômica, e após o sucesso o utilizador é redirecionado para o detalhe do encaminhamento.
- Eliminação (encaminhamentos_eliminar) - A eliminação de um encaminhamento é simples, uma vez que não existem outras tabelas que dependam dele (a base de dados está configurada com ON DELETE RESTRICT, mas não há chaves estrangeiras a apontar para ENCAMINHAMENTOS_REDE). A view verifica primeiro se o encaminhamento existe; em caso afirmativo, executa um DELETE em SQL puro (também poderia ser ORM) dentro de uma transação atômica. Após a eliminação, o utilizador é redirecionado para a lista de encaminhamentos, com uma mensagem de sucesso.
- Contactos: O módulo de contactos destina-se exclusivamente aos utentes idosos, tendo como objetivo registar todas as interações realizadas pela equipa de intervenção social com cada idoso. As informações recolhidas incluem o tipo de contacto (visita domiciliária, chamada telefónica, atividade em grupo, atendimento no balcão, videochamada, contacto com familiar), a frequência do acompanhamento (único, semanal, quinzenal, mensal, trimestral), a data e hora do contacto, e observações livres. Este registo sistemático permite não só manter um histórico de acompanhamento, como também identificar situações de isolamento (falta de contacto prolongada) e gerar alertas automáticos. As operações CRUD foram desenhadas para serem intuitivas e eficientes, mesmo quando existem centenas de idosos cadastrados.
 - Listagem (listar_contactos) - A página de listagem de contactos apresenta uma tabela paginada (10 registos por página) com as colunas: ID, nome do

idoso, tipo de contacto, data/hora, frequência, observações (truncadas a 50 caracteres), administrador responsável e ações (ver, editar, eliminar). A ordenação é decrescente por data de contacto, mostrando em primeiro lugar os contactos mais recentes. A consulta SQL principal envolve as tabelas CONTACTO, UTENTES_IDOSOS, UTENTES, ADMINISTRADORES, TIPO_CONTACTO e FREQUENCIA_CONTACTO, garantindo que todos os dados descritivos são obtidos através de JOIN. A paginação é implementada com o Paginator do Django. Não foi implementado um filtro específico, mas a paginação assegura uma boa experiência mesmo com muitos registos.

- Detalhe (detalhe_contacto) - A página de detalhe de um contacto mostra toda a informação associada, organizada em dois cartões. O primeiro cartão contém os dados do contacto: ID, nome do idoso (com link para a ficha de detalhe do idoso), tipo de contacto, data/hora, frequência, observações e administrador responsável. O segundo cartão apresenta um resumo do idoso na data do contacto: nome, idade (calculada na data do contacto, recorrendo ao snapshot histórico mais recente até essa data), sexo, zona, “vive sozinho”, contacto de emergência e familiar/cuidador de referência. Esta informação é obtida através da tabela de histórico historico_idoso, seleccionando o snapshot mais recente anterior ou igual à data do contacto, garantindo a coerência histórica. Desta forma, mesmo que o idoso tenha alterado a sua situação habitacional ou de saúde posteriormente, o relato do contacto reflete o estado real no momento da interação.
- Criação (contactos_novo) - O formulário de criação de um novo contacto está dividido em duas partes. A primeira parte consiste na seleção do idoso. Para lidar com um número elevado de idosos (centenas ou milhares), optou-se por um <select> estilizado com a biblioteca Select2 (carregamento completo, não AJAX), que oferece pesquisa local por nome, código ou zona. As opções são carregadas a partir da lista de todos os idosos, com a apresentação de informação suficiente (código, nome, idade, zona) para uma escolha informada. A segunda parte contém os restantes campos: tipo de contacto (obrigatório, seleccionado a partir de um dropdown com valores pré-definidos), frequência de contacto (opcional, dropdown), data e hora

(obrigatória, campo datetime-local) e observações (campo de texto livre). São aplicadas validações simples: idoso obrigatório, tipo de contacto obrigatório, data obrigatória. A criação é feita através do ORM, envolvida numa transação atómica, uma vez que a tabela CONTACTO tem uma chave primária simples. Após a criação, o utilizador é redirecionado para o detalhe do contacto.

- Edição (contactos_editar) - A edição de um contacto carrega previamente todos os dados atuais para o formulário, incluindo o idoso selecionado (que aparece pré-selecionado no Select2) e os restantes campos (tipo de contacto, frequência, data/hora, observações). O técnico pode alterar qualquer um destes campos, incluindo o idoso associado. As validações são idênticas às da criação. A atualização é feita através do ORM, envolvida numa transação atómica. Antes de guardar, é importante garantir que a data do contacto seja respeitada e que não haja inconsistências. Após a atualização, o utilizador é redirecionado para o detalhe do contacto.
- Eliminação (contactos_eliminar) - A eliminação de um contacto é uma operação simples, uma vez que não existem outras tabelas que dependam de CONTACTO (as chaves estrangeiras estão configuradas com ON DELETE RESTRICT, mas nenhuma outra tabela referencia CONTACTO). A view verifica primeiro se o contacto existe; em caso afirmativo, executa um DELETE em SQL puro (ou ORM) dentro de uma transação atómica. Após a eliminação, o utilizador é redirecionado para a lista de contactos, com uma mensagem de sucesso. Note-se que a eliminação de um contacto não afecta o histórico do idoso, uma vez que o histórico é mantido separadamente.
- Dashboard: O dashboard apresenta um conjunto de indicadores de gestão e análise estatística, obtidos através de vistas SQL pré-definidas. Os dados são carregados com raw SQL e passados ao template em formato JSON para alimentar gráficos (Chart.js). Os principais indicadores incluem: totais gerais (idosos, socioeconómicos, intervenções, contactos), evolução mensal de intervenções e contactos, problemáticas mais frequentes (ativas), evolução temporal de problemáticas (novos casos vs. ativos), taxa de reincidência por problemática, tipos de intervenção e programas mais utilizados, risco social, perfil etário e género de utentes, autonomia

e isolamento dos idosos, idosos em situação de risco (isolamento elevado + vive sozinho, quedas repetidas, falta de rede de suporte, visitas prioritárias), respostas mais utilizadas nos encaminhamentos e distribuição de utentes por zona e risco social. O dashboard é actualizado em tempo real, uma vez que as vistas SQL refletem sempre os dados mais recentes.

- **Relatórios PDF:** A plataforma permite gerar relatórios mensais e anuais em formato PDF, utilizando a biblioteca ReportLab. O utilizador introduz o ano e, opcionalmente, o mês num formulário no dashboard. A view correspondente calcula as datas de início e fim do período e executa consultas SQL filtradas por essas datas (utilizando `data_criacao` para utentes, `data_inicio` para problemáticas, `data_resposta` para encaminhamentos, etc.). Todos os indicadores apresentados no dashboard são incluídos no PDF, mas com a particularidade de serem filtrados pelo período seleccionado (os dados globais são devidamente assinalados). O PDF é gerado em modo paisagem, com tabelas formatadas, parágrafos estilizados e uma nota de rodapé com a data/hora de geração. O ficheiro é devolvido ao navegador para download.
- **Histórico de Utes:** Para garantir a rastreabilidade das alterações, a plataforma guarda automaticamente um snapshot do estado completo de cada utente (socioeconómico ou idoso) sempre que é criado ou editado. Esses snapshots são armazenados em tabelas de histórico (`historico_utente_socioeconomico` e `historico_idoso`) utilizando o tipo JSONB do PostgreSQL. A estrutura guardada inclui todos os dados principais, as listas de intervenções, problemáticas (com datas e estados), membros do agregado, despesas, escalões, e, no caso dos idosos, também as actividades, doenças, barreiras, quedas e contactos. A função de guarda é chamada no início da transacção de edição (antes das alterações) e após a criação. O administrador pode consultar o histórico através de uma página de listagem (com data e utilizador que efetuou a alteração) e visualizar qualquer versão anterior num template semelhante ao detalhe, mas com um aviso claro de que se trata de uma versão antiga. É ainda possível eliminar registos históricos individualmente ou limpar todo o histórico de um utente. Esta funcionalidade é essencial para auditoria e para recuperação de informação em caso de erro.

4.3.5 Integração com o Frontend

A integração entre o backend e o frontend foi realizada através do motor de templates do Django, que gera dinamicamente as páginas HTML a partir dos dados fornecidos pelas views. Os dados são passados no contexto e inseridos nos templates utilizando as tags e filters nativos da linguagem de templates (ex: `{{ variavel }}`, `{% for %}`, `{{ valor|date:"d/m/Y" }}`). Para assegurar uma experiência de utilizador responsiva e moderna, foram adotadas as seguintes práticas:

- Bootstrap 5 – utilizado para a estruturação das páginas (grelha, cartões, formulários, tabelas) e para a adaptação a diferentes tamanhos de ecrã (design responsivo).
- Ícones - a biblioteca Bootstrap Icons fornece os ícones usados nos botões e cabeçalhos, melhorando a comunicação visual.
- Gráficos – no dashboard, os dados agregados são convertidos em JSON no backend e passados ao template através da tag `json_script`, que os torna acessíveis ao JavaScript. A biblioteca Chart.js consome esses dados para desenhar gráficos de barras, linhas e pizza.
- Dropdowns pesquisáveis – nos formulários de edição de utentes socioeconómicos, os dropdowns de intervenções (para associar resolução de problemáticas) foram implementados com a biblioteca Select2, que permite pesquisar entre muitas opções e navegar por teclado, melhorando a usabilidade.
- Campos dinâmicos – foram criadas tabelas com linhas adicionáveis/removíveis para membros do agregado, despesas, escalões de rendimento e quedas, recorrendo a JavaScript vanilla para manipulação do DOM.
- Formulários e validação – os formulários são submetidos por método POST com proteção CSRF. O Django valida os dados no servidor e, em caso de erro, reexibe o formulário com as mensagens apropriadas. A validação client-side (atributos `required`, `type="date"`, etc.) complementa a experiência, mas não substitui a validação no servidor.
- Paginação e filtros – as listagens utilizam o Paginator do Django e mantêm os parâmetros de filtro (ex: `?nome=...`) nos links de navegação, garantindo a persistência das pesquisas.
- Redireccionamentos – após operações de criação, edição ou eliminação, o utilizador é redirecionado para a página de lista ou de detalhe, evitando reenvios acidentais de formulários.

Esta abordagem, baseada maioritariamente em renderização no servidor, simplifica o desenvolvimento e mantém o código frontend leve e de fácil manutenção, sem necessidade de frameworks JavaScript pesadas. A comunicação assíncrona (AJAX) foi reservada apenas para funcionalidades específicas que beneficiariam de melhorias de usabilidade (como a pesquisa em dropdowns com muitos dados), mas na prática o volume de dados atual não exigiu a sua implementação.

5 Resultados Obtidos

Neste capítulo apresentam-se os principais resultados alcançados com o desenvolvimento da plataforma, bem como um conjunto de propostas de melhoria para a organização, com base na experiência adquirida durante o estágio.

5.1 Síntese dos resultados alcançados

No final do estágio, foi entregue uma plataforma web funcional que permite à equipa de intervenção social gerir de forma integrada os utentes (socioeconómicos e idosos), as intervenções, os encaminhamentos, os contactos e os indicadores de gestão. Os principais resultados podem ser resumidos do seguinte modo:

- Centralização da informação: todos os registos anteriormente dispersos em folhas de cálculo e documentos físicos passaram a estar disponíveis numa única base de dados, com acesso rápido e seguro.
- Redução do tempo de consulta: a pesquisa por nome e a paginação nas listagens permitem localizar um utente ou uma intervenção em poucos segundos, ao contrário dos métodos manuais anteriores.
- Geração automática de relatórios – o administrador pode obter, com apenas dois cliques, relatórios mensais ou anuais em formato PDF, contendo dezenas de indicadores estatísticos, sem necessidade de tratamento manual de dados.

A plataforma foi testada com dados reais anonimizados fornecidos pela entidade de acolhimento, tendo sido validados todos os fluxos principais (criação, edição, eliminação, consulta e geração de relatórios). A equipa técnica manifestou satisfação com a usabilidade e com a rapidez de resposta do sistema.

5.2 Propostas de melhoria para a organização

Apesar do sucesso do projeto, foram identificadas oportunidades de evolução que poderão trazer valor adicional à entidade:

- Integração com outras bases de dados: a plataforma poderia importar dados de utentes já existentes em sistemas autárquicos ou da Segurança Social, evitando a duplicação de registos.
- Alertas automáticos – em vez de apenas visualizar os alertas no dashboard, a plataforma poderiam enviar notificações automáticas para os técnicos responsáveis, melhorando a capacidade de resposta.
- Acesso móvel – o desenvolvimento de uma versão otimizada para telemóveis (ou de uma aplicação móvel nativa) permitiria aos técnicos registar contactos e consultar informação durante visitas domiciliárias, mesmo sem acesso a um computador.
- Maior granularidade de permissões – atualmente todos os administradores têm acesso total. A introdução de perfis diferenciados (ex: técnico de campo, coordenador, administrador) aumentaria a segurança e a conformidade com o RGPD.
- Geração de gráficos nos relatórios PDF – os relatórios poderiam incluir representações gráficas dos principais indicadores (evolução, distribuição, etc.), tornando os mais intuitivos para apresentação a entidades externas.
- Cálculo automático do tempo médio de resposta – embora o campo TEMPO_RESPOSTA esteja presente, o sistema faria automaticamente.

Estas propostas não foram implementadas durante o estágio por limitação de tempo, mas constituem um roteiro claro para a evolução futura da plataforma.

6 Conclusão

Neste capítulo faz-se um balanço global do trabalho realizado, avaliando-se a concretização dos objetivos definidos, as principais aprendizagens e as perspetivas de desenvolvimento futuro.

6.1 Principais Resultados e Contribuições

O estágio permitiu desenvolver uma plataforma completa de gestão da intervenção social comunitária, respondendo às necessidades iniciais da entidade de acolhimento. Todos os objetivos específicos foram atingidos: conceção da base de dados, implementação dos módulos CRUD para utentes (socioeconómicos e idosos), gestão de intervenções, encaminhamentos e contactos, criação de um dashboard analítico, geração de relatórios PDF e sistema de histórico de alterações.

O principal contributo do projeto é a disponibilização de uma ferramenta que centraliza a informação, reduz a redundância administrativa e melhora a capacidade de monitorização dos indicadores de risco. A plataforma já demonstrou potencial para fundamentar decisões estratégicas (ex: alocação de recursos a zonas com maior pressão social) e para agilizar o trabalho diário dos técnicos. Além disso, o sistema de histórico constitui uma mais-valia para a auditoria e para a rastreabilidade das intervenções, alinhando-se com as exigências do RGPD.

6.2 Trabalho Futuro

Apesar de a plataforma estar funcional e pronta a ser utilizada, várias linhas de evolução podem ser seguidas para ampliar o seu valor:

- Integração com sistemas externos – nomeadamente com as plataformas da Segurança Social ou dos serviços de saúde, para troca automática de informação sobre utentes.

- Aplicação móvel – desenvolvimento de uma versão mobile (ou de um PWA) que permita o registo de contactos e quedas em tempo real durante as visitas domiciliárias.
- Inteligência artificial – a introdução de modelos preditivos (ex: risco de isolamento ou de reincidência de problemáticas) poderia ajudar a equipa a intervir preventivamente.
- Personalização de relatórios – permitir ao administrador escolher quais os indicadores a incluir no PDF e em que ordem.

6.3 Aprendizagem e Reflexão Pessoal

A realização deste estágio constituiu uma experiência formativa de grande valor. Do ponto de vista técnico, consolidaram-se conhecimentos em Django, PostgreSQL, SQL puro, ReportLab e integração front-end, bem como em boas práticas de segurança e transações atómicas. A nível de metodologia, a abordagem ágil (sprints semanais) revelou-se eficaz para gerir o tempo e responder a requisitos em evolução. Os aspetos mais positivos incluem a autonomia concedida para tomar decisões de arquitetura e a satisfação de ver a plataforma a ser bem recebida pela entidade. As principais dificuldades residiram na gestão do histórico de alterações (especialmente para os idosos, devido ao grande número de tabelas) e na otimização das consultas analíticas – obstáculos que foram ultrapassados com pesquisa e experimentação. No futuro, esta experiência será um forte contributo para a minha carreira profissional, demonstrando capacidade de desenvolver um sistema completo, com incidência na gestão de dados sensíveis e na produção de relatórios.

Referências Bibliográficas

- [1] C. Santiago, N. de Lima Veras, A. Aragão, D. Carvalho, e L. Amaral, «Desenvolvimento de sistemas Web orientado a reuso com Python, Django e Bootstrap», 2020, pp. 97–120. doi: 10.5753/sbc.11.5.5.
- [2] K. S. Duisebekova, R. Khabirov, e A. Zholzhan, «DJANGO AS SECURE WEB-FRAMEWORK IN PRACTICE», *Вестник КазАТК*, vol. 116, n.º 1, pp. 275–281, mar. 2021, doi: 10.52167/1609-1817-2021-116-1-275-281.
- [3] «Django Tutorial Parte 5: Criando nossa home page - Aprendendo desenvolvimento web | MDN», MDN Web Docs. Acedido: 28 de junho de 2026. [Em linha]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Extensions/Server-side/Django/Home_page
- [4] «Django Tutorial: Python Web Development». Acedido: 28 de junho de 2026. [Em linha]. Disponível em: <https://www.datacamp.com/tutorial/web-development-django>
- [5] S. Chen, S. Ahmmed, K. Lal, e C. Deming, «Django Web Development Framework: Powering the Modern Web», *Am. J. Trade Policy*, vol. 7, n.º 3, pp. 99–106, dez. 2020, doi: 10.18034/ajtp.v7i3.675.
- [6] «Django Web Framework (Python) - Learn web development | MDN», MDN Web Docs. Acedido: 28 de junho de 2026. [Em linha]. Disponível em: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Django
- [7] S. Dauzon, A. Bendoraitis, e A. Ravindran, *Django: Web Development with Python*. Packt Publishing Ltd, 2016.
- [8] «O que é o Django? – Explicação sobre o software Django – AWS», Amazon Web Services, Inc. Acedido: 28 de junho de 2026. [Em linha]. Disponível em: <https://aws.amazon.com/pt/what-is/django/>
- [9] «Python Web Development With Django», GeeksforGeeks. Acedido: 28 de junho de 2026. [Em linha]. Disponível em: <https://www.geeksforgeeks.org/python/python-web-development-django/>
- [10] J. Sunte, «Recent Trends in Computer Graphics and Multimedia Technology Overview of Django web application framework for Python», vol. 8, pp. 39–42, jan. 2026, doi: 10.5281/zenodo.18276725.
- [11] A. Holovaty e J. Kaplan-Moss, *The definitive guide to Django: Web development done right*, 2nd ed. em *The experts's voice in Web development*. Berkeley, CA : New York: Apress ; Distributed to the book trade worldwide by Springer-Verlag, 2009.
- [12] «W3Schools.com». Acedido: 28 de junho de 2026. [Em linha]. Disponível em: https://www.w3schools.com/django/django_intro.php
- [13] «What Is the Django Web Framework?», The JetBrains Blog. Acedido: 28 de junho de 2026. [Em linha]. Disponível em: <https://blog.jetbrains.com/pycharm/2023/11/what-is-the-django-web-framework/>